Object-oriented programming

CSC148, INTRODUCTION TO COMPUTER SCIENC

Related Reading: 3.1 Intro. to Object-Oriented Programming

This Lecture: `class` and composition!

By the end of lecture, you should be able to:

- Python class
  - Define, identify & produce examples of key terms (next slide)
  - Program a Python class, including **initializer**, **attributes**, and **methods**
  - Translate Python code that uses classes into a **memory model**
  - Determine the values of variables and instance attributes from given code and/or memory model involving classes
- Composition
  - Define **composition**, and identify & create code that uses composition
  - Explain whether composition is appropriate for a given problem

Key terms and phrases

class

instance (of a class)

instance attribute

method

initializer

dot notation

self

Key terms and phrases

class

instance (of a class)

instance attribute

method

initializer

dot notation

self

```
class Tweet:
    ...
```

Key terms and phrases

class

instance (of a class)

instance attribute

method

initializer

dot notation

self

```
class Tweet:
    def __init__(self, ...) -> None:
        ...

t = Tweet(...) # This calls the initializer
```

Key terms and phrases

class

instance (of a class)

instance attribute

method

initializer

dot notation

self

```
class Tweet:
    def __init__(self, ...) -> None:
        ...

t = Tweet(...) # This calls the initializer

t.likes
```

Key terms and phrases

class

instance (of a class)

instance attribute

method

initializer

dot notation

self

```
class Tweet:
    def __init__(self, ...) -> None:
        ...

t = Tweet(...) # This calls the initializer

t.likes          t.like()
```

Initializer __init__

```
class Tweet:
    def __init__(self, ...) -> None:
t = Tweet(...)
```

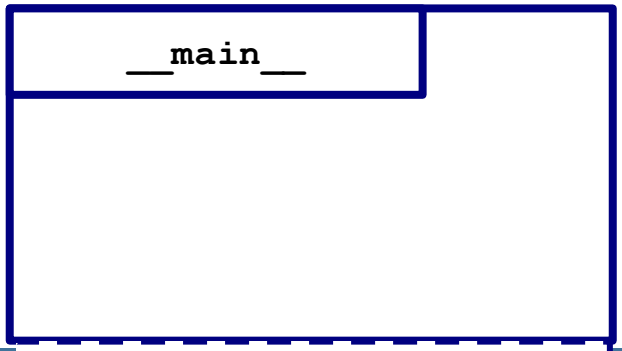
- When we create an instance by calling Tweet(...), Python calls __init__ for us
- Returns None because there is no return statement
- Behind the scenes:
  - Python creates an empty Tweet object
  - Python calls __init__ for us
  - Returns the Tweet object it created

```python
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

```python
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

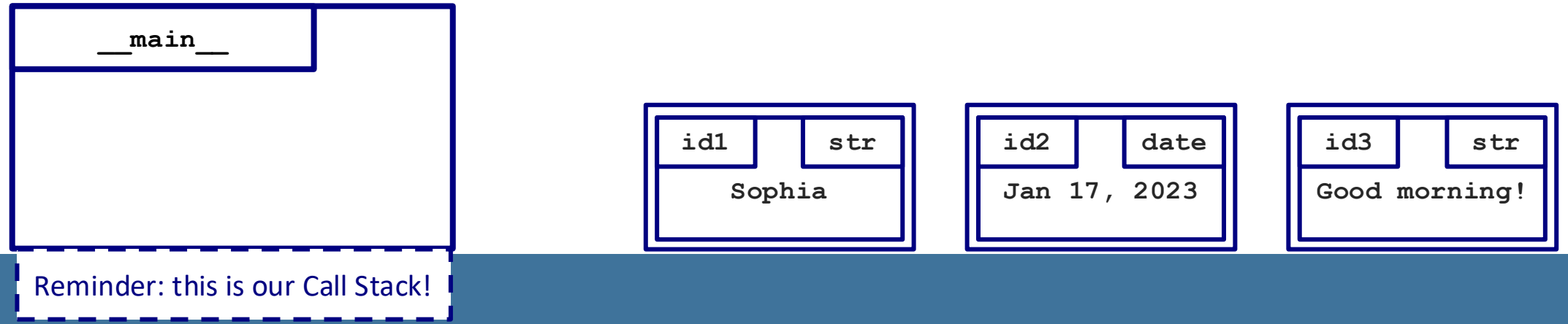
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```



Reminder: this is our Call Stack!

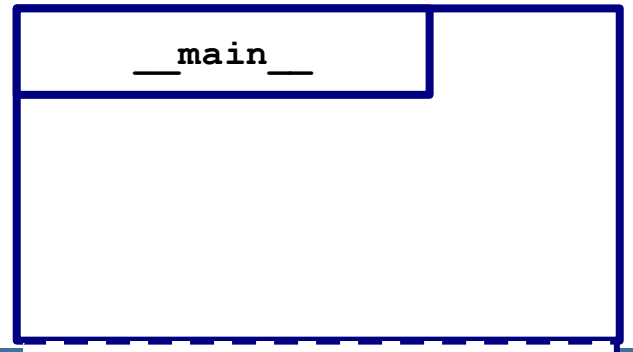
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

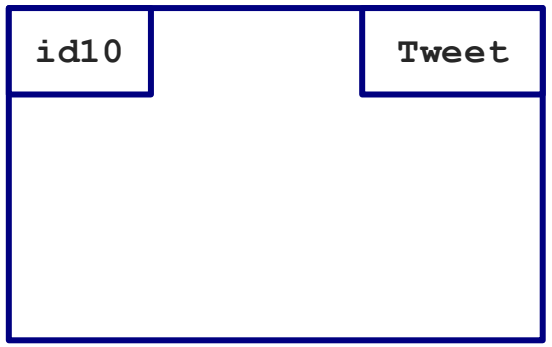
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

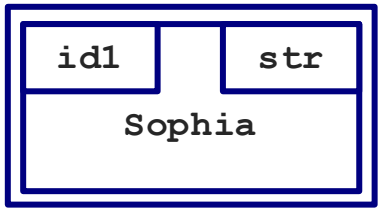


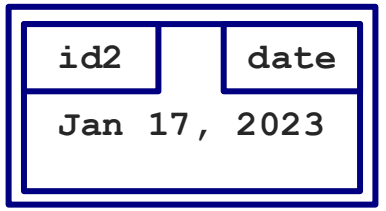
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

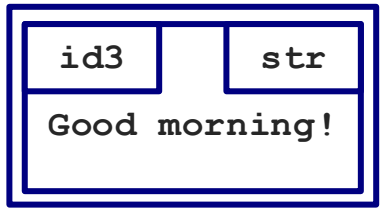
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







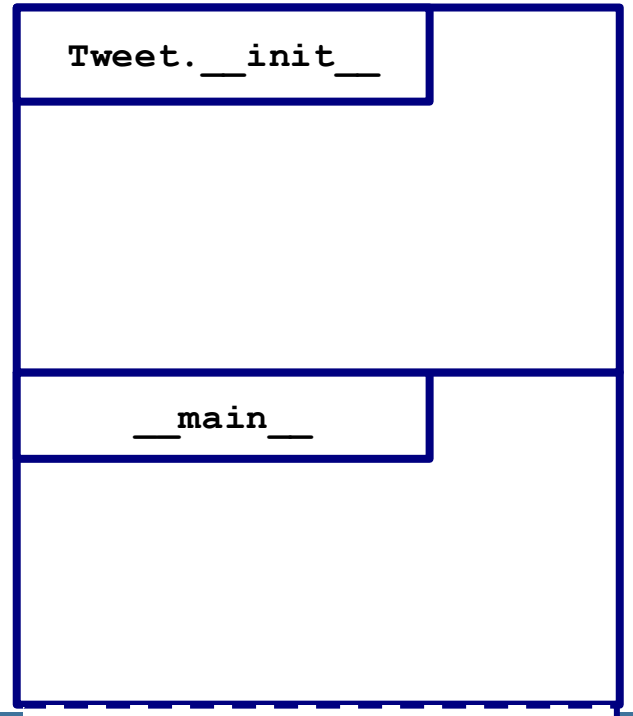


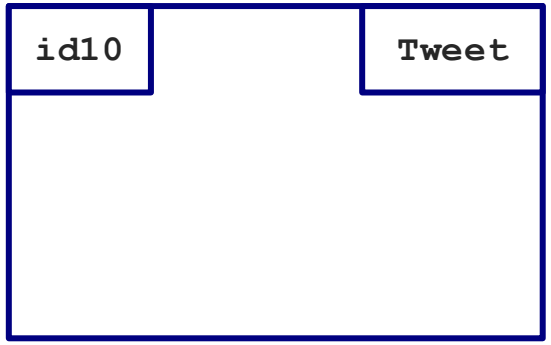


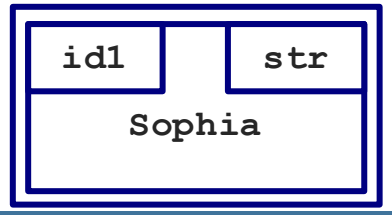
Reminder: this is our Call Stack!

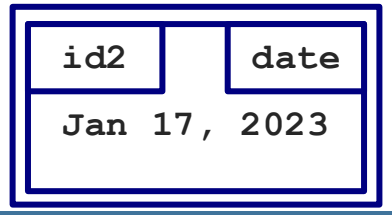
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

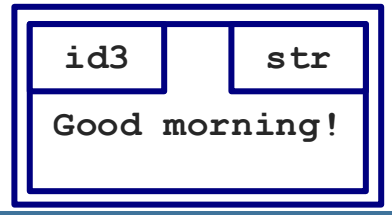
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







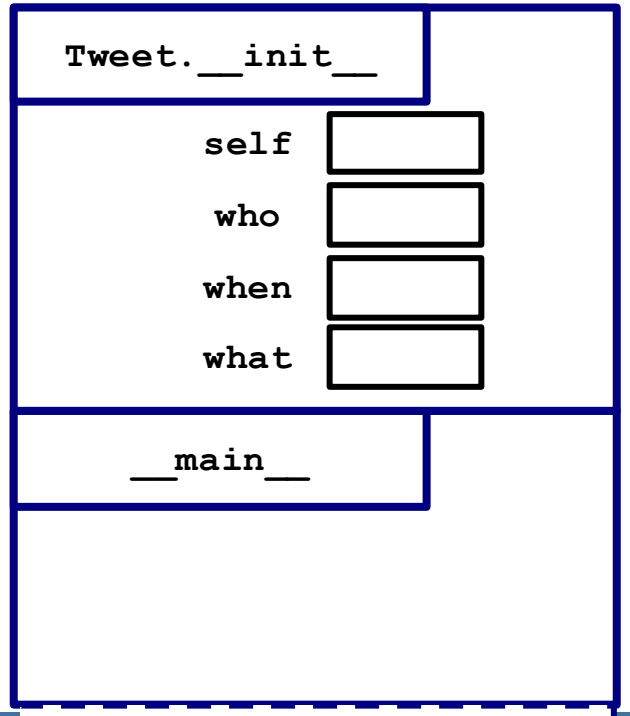


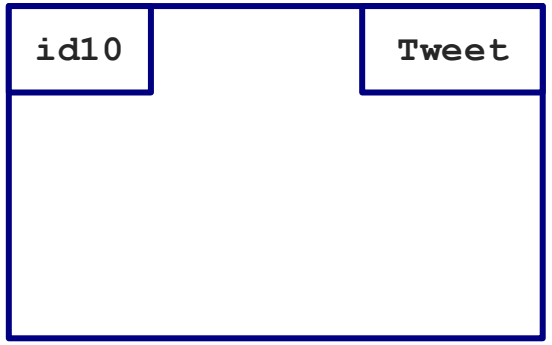


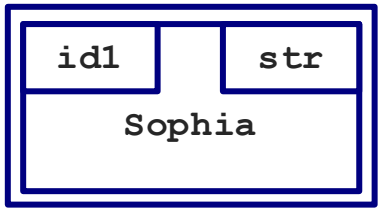
Reminder: this is our Call Stack!

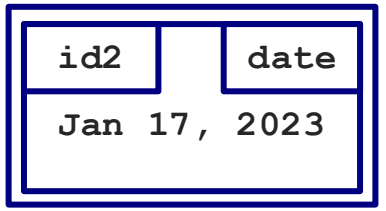
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

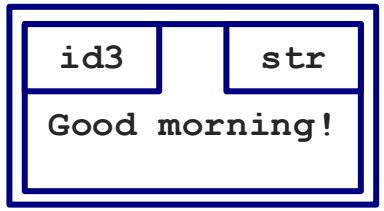
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







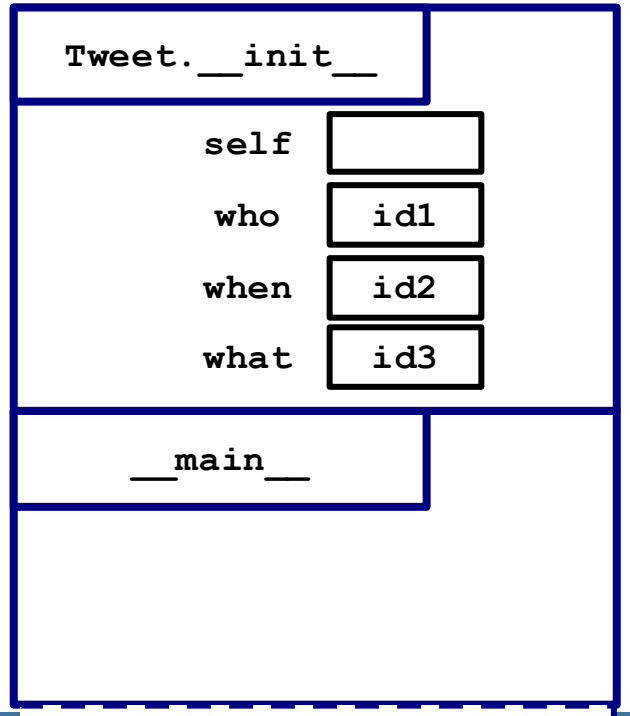


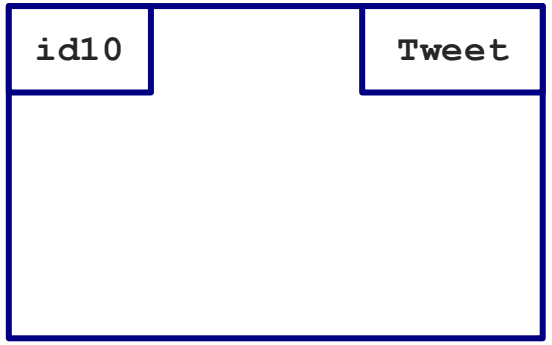


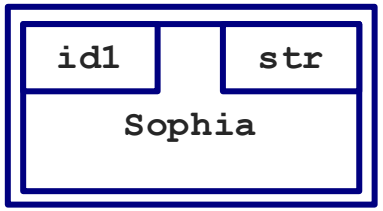
Reminder: this is our Call Stack!

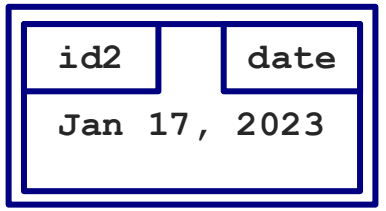
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

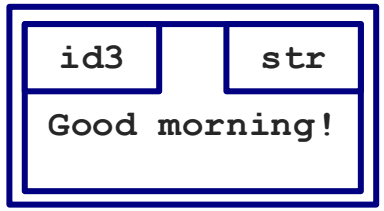
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







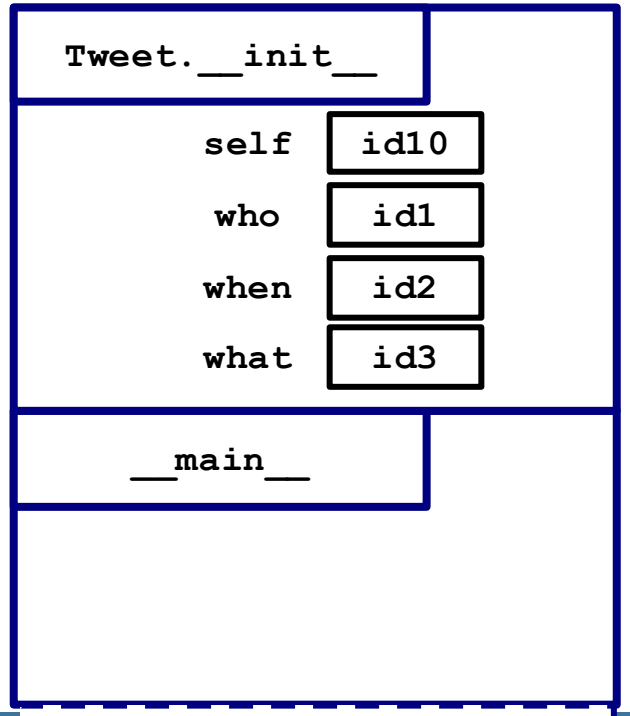


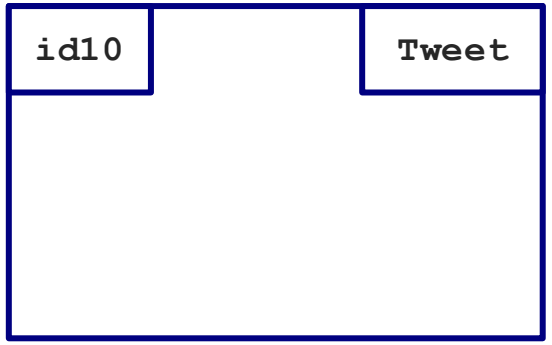


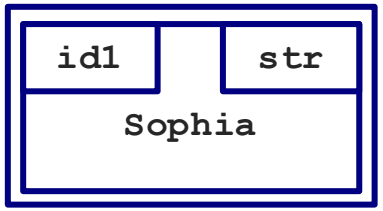
Reminder: this is our Call Stack!

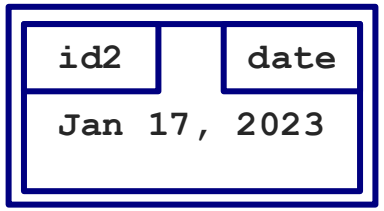
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

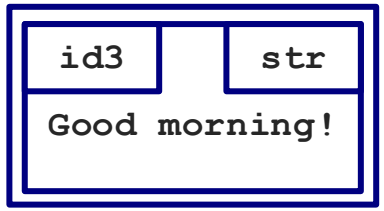
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







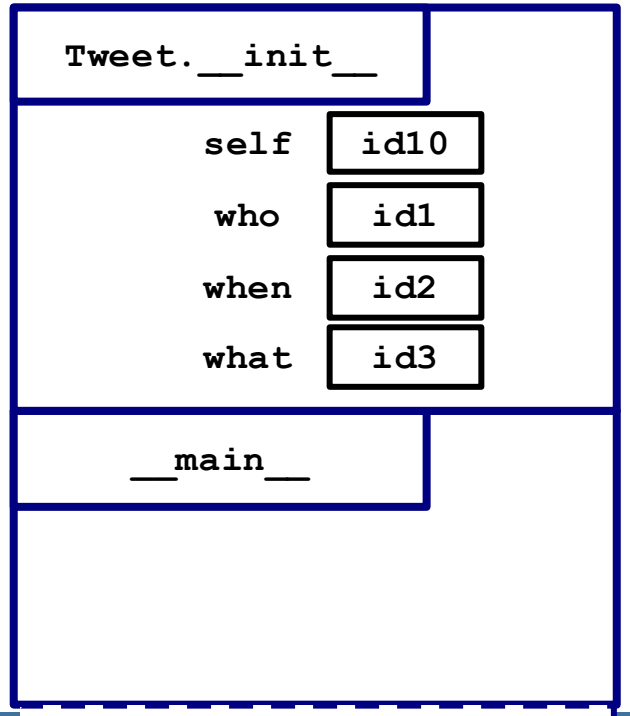


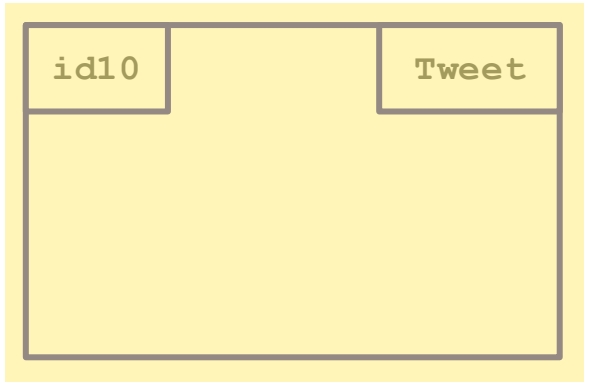


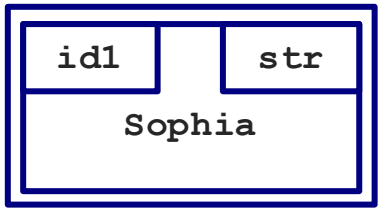
Reminder: this is our Call Stack!

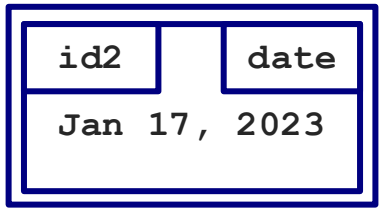
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

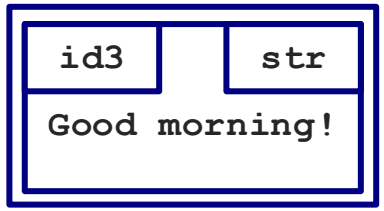
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







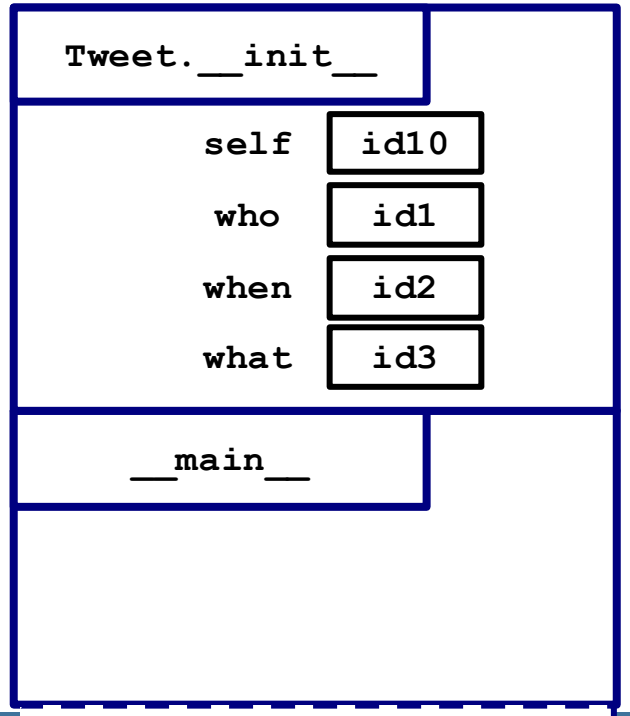


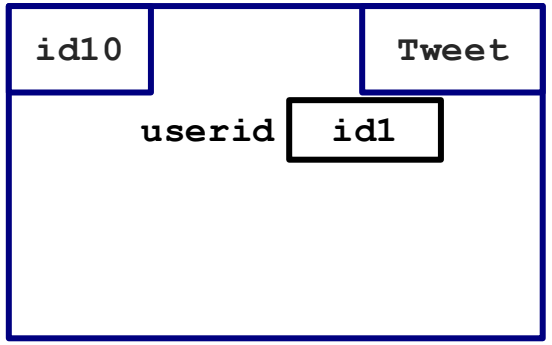


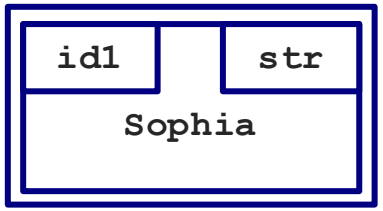
Reminder: this is our Call Stack!

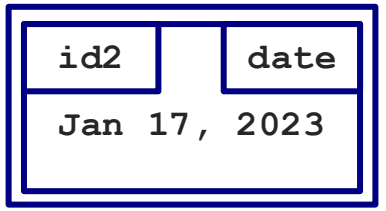
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

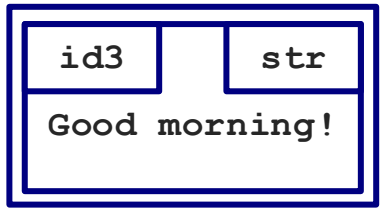
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







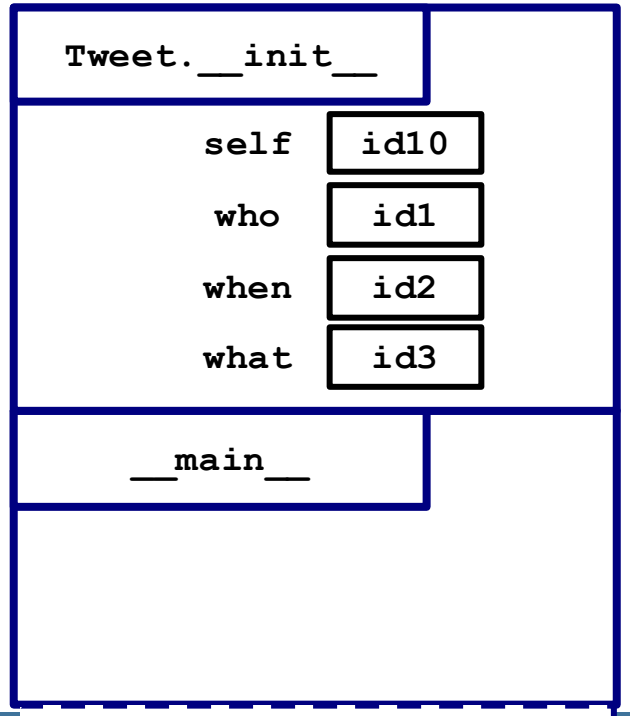


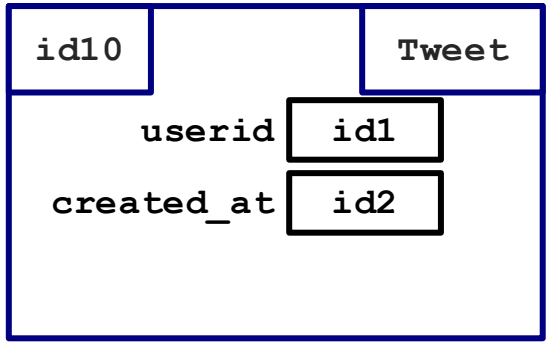


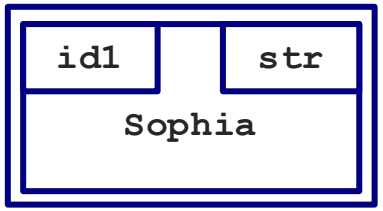
Reminder: this is our Call Stack!

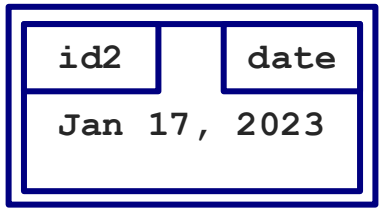
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

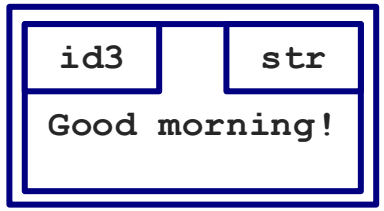
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







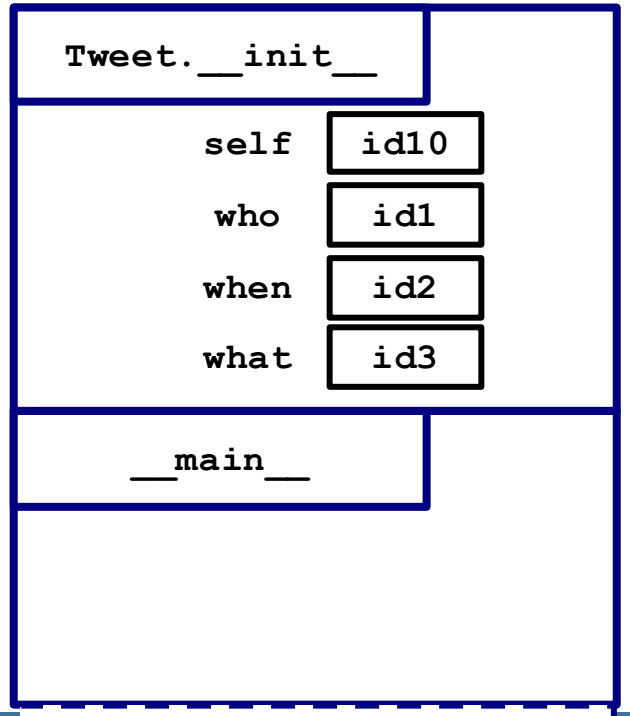


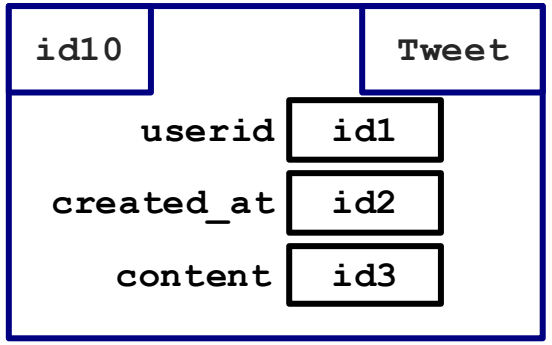


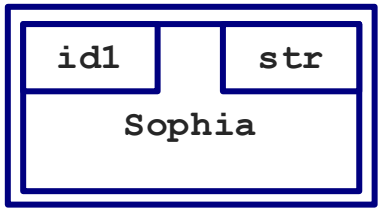
Reminder: this is our Call Stack!

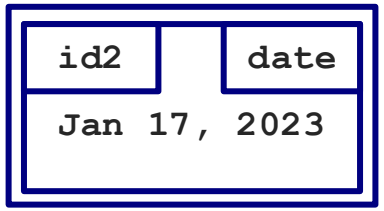
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

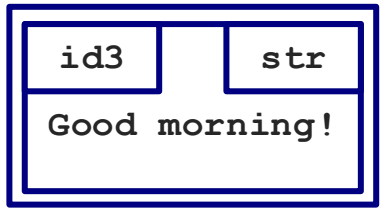
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```







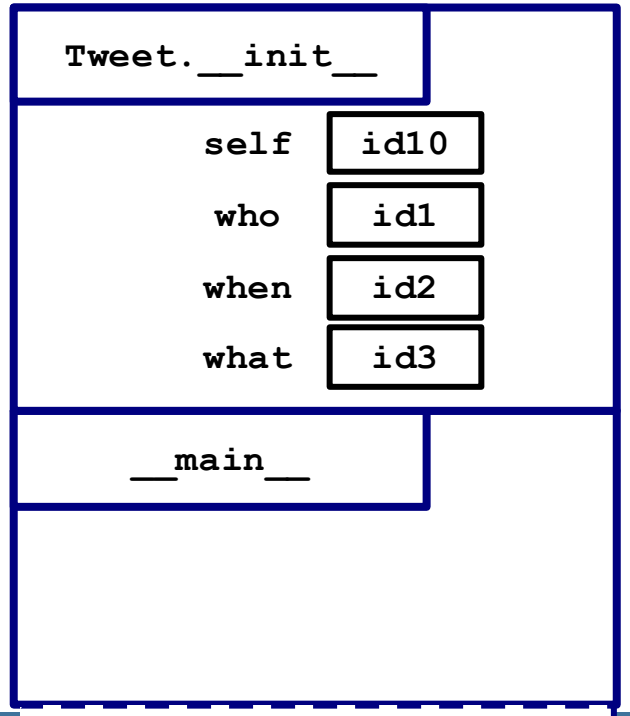


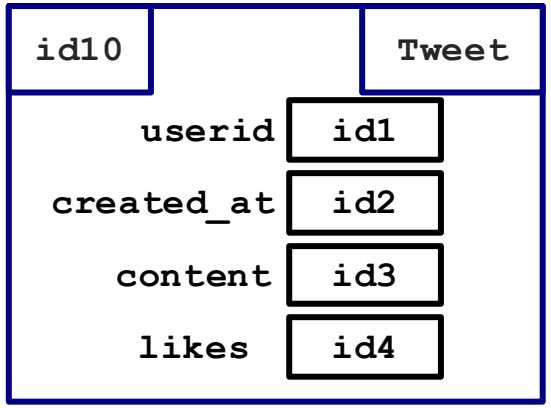


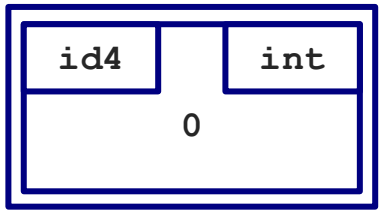
Reminder: this is our Call Stack!

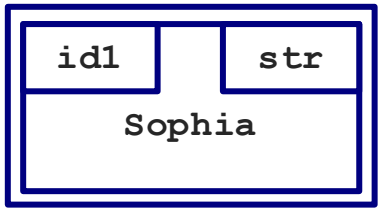
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

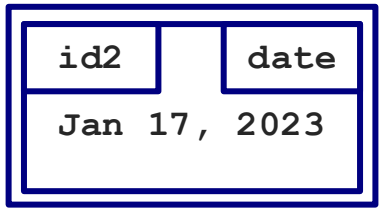
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

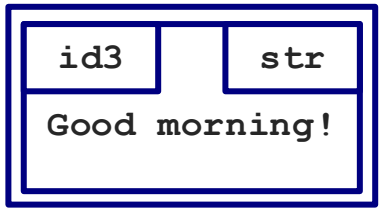








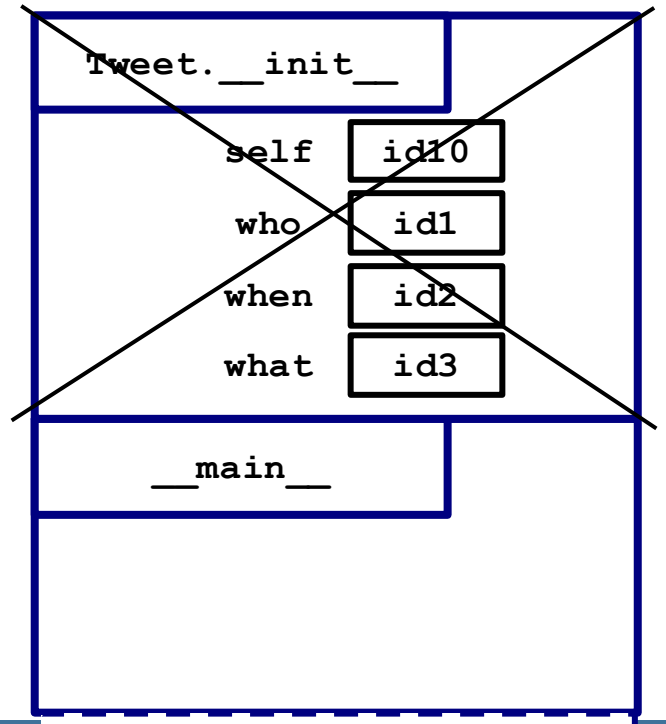


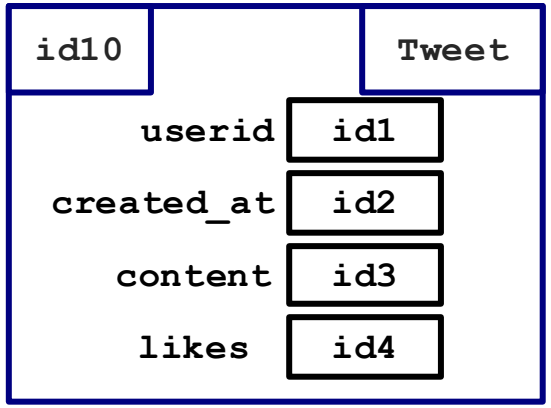


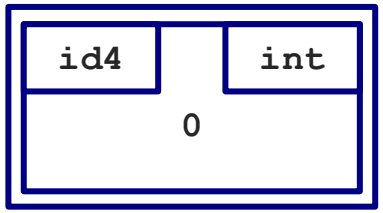
Reminder: this is our Call Stack!

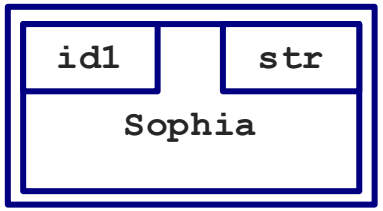
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

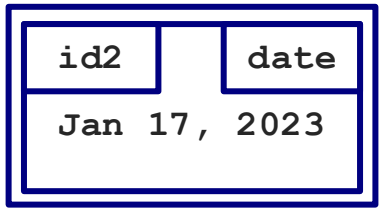
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

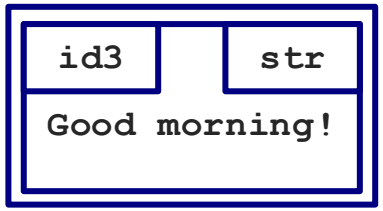








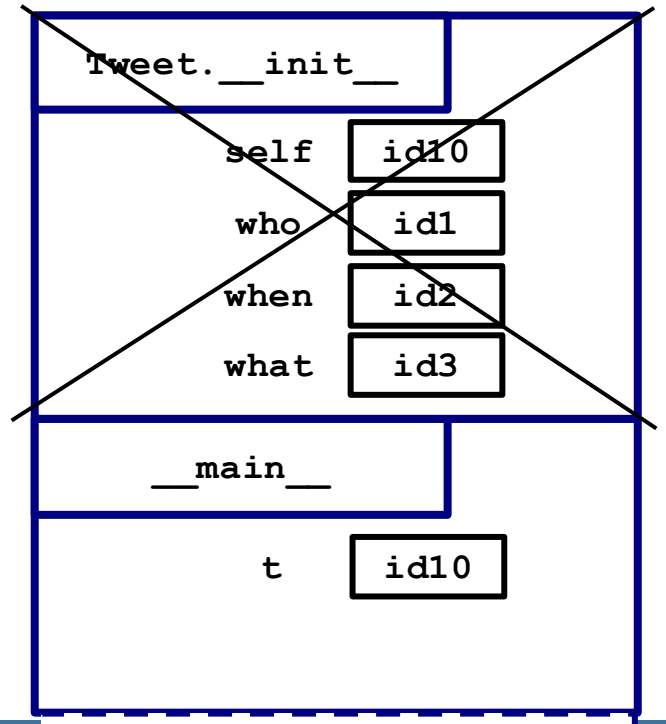


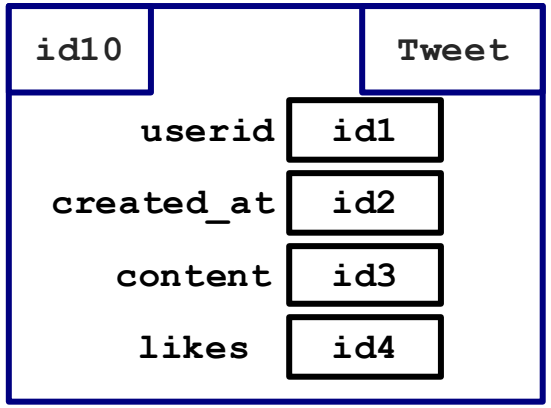


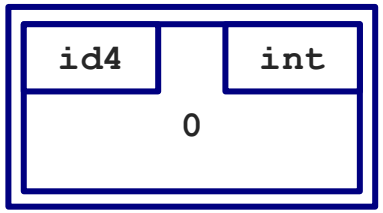
Reminder: this is our Call Stack!

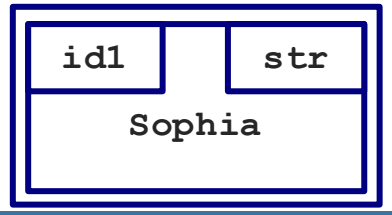
```
class Tweet:
    def __init__(self, who: str,
                 when: date, what: str) -> None:
        self.userid = who
        self.created_at = when
        self.content = what
        self.likes = 0

    def like(self, n: int) -> None:
        self.likes += n
```

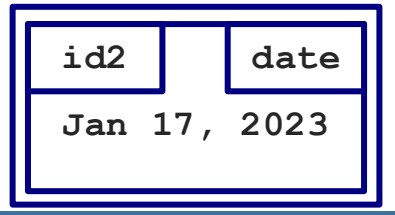
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

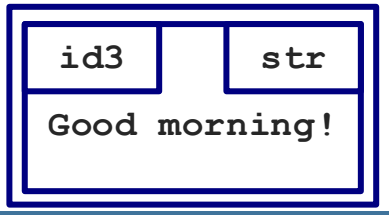








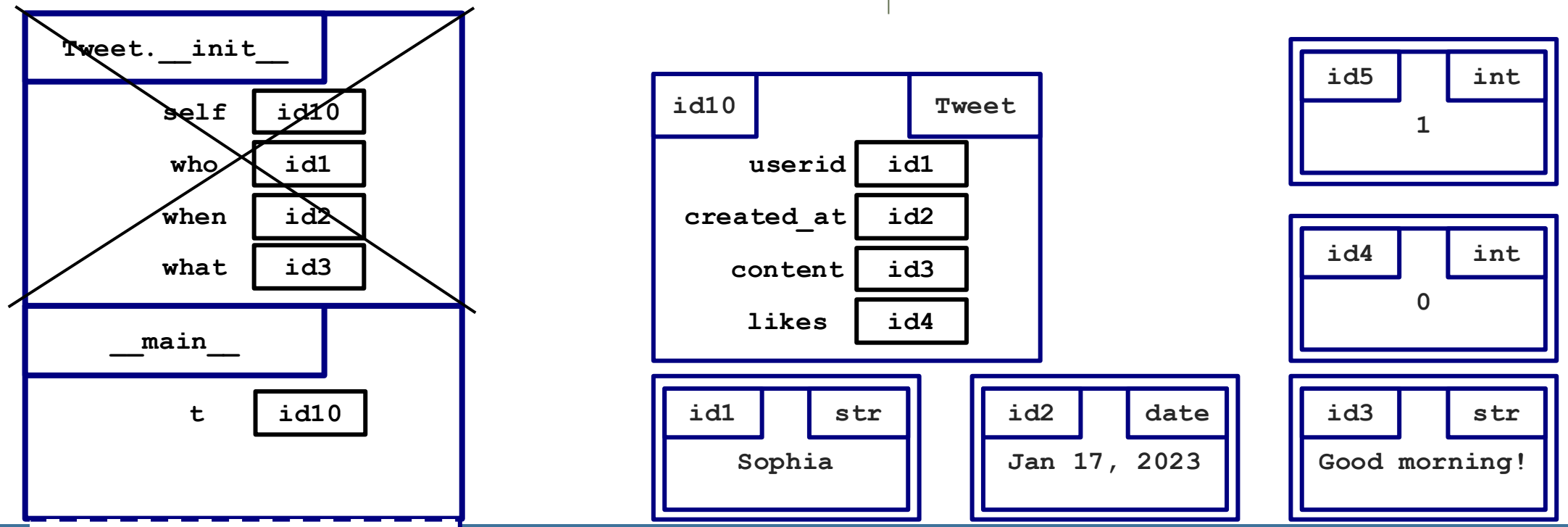




Reminder: this is our Call Stack!

```
class Tweet:
    ...
        def like(self, n: int) -> None:
            self.likes += n
```

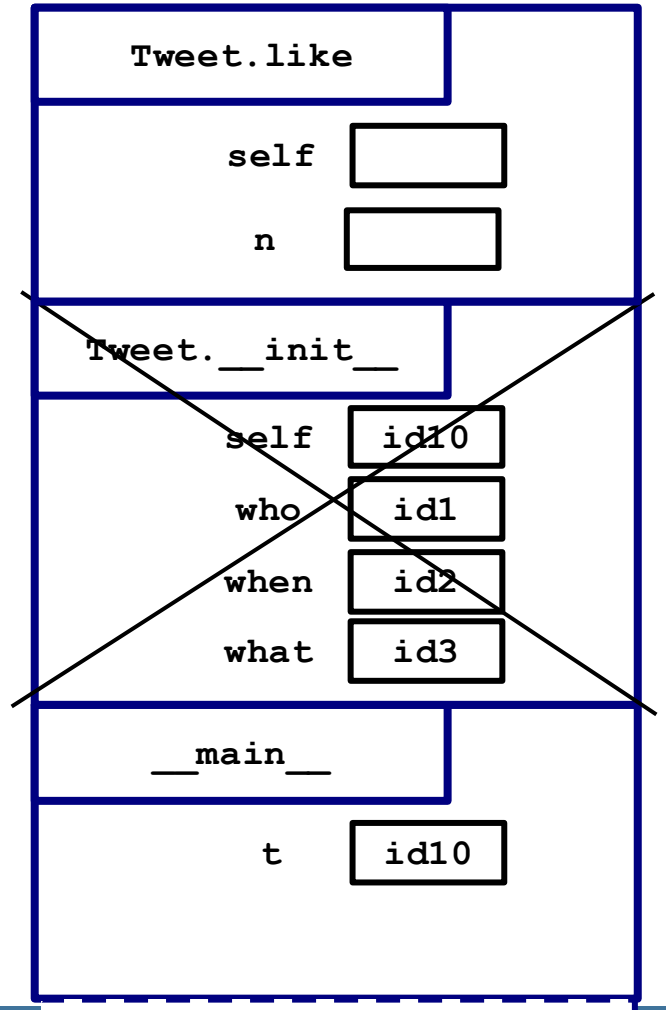
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
➡   t.like(1)
```

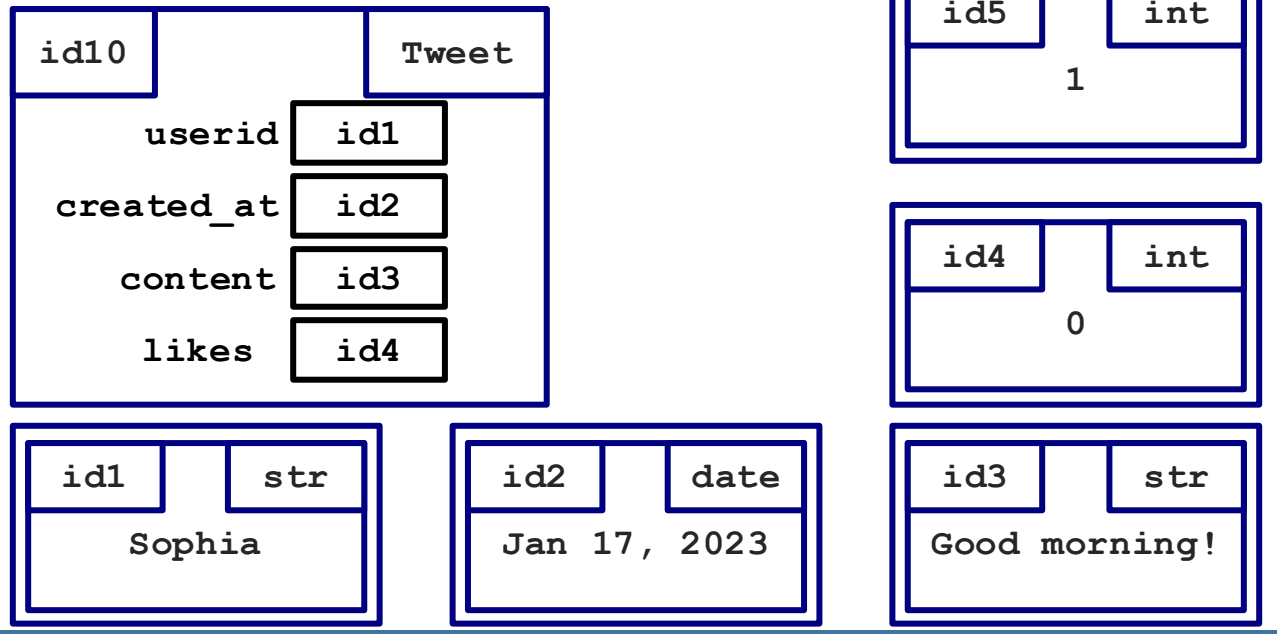


Reminder: this is our Call Stack!

```
class Tweet:
    ...
    def like(self, n: int) -> None:
        self.likes += n
```

```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

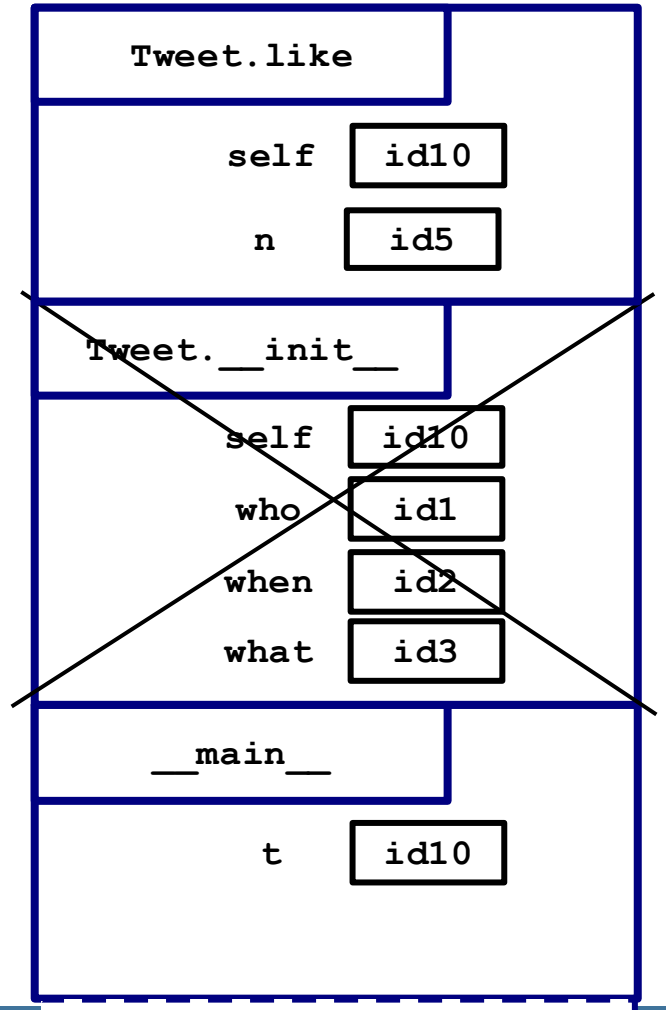


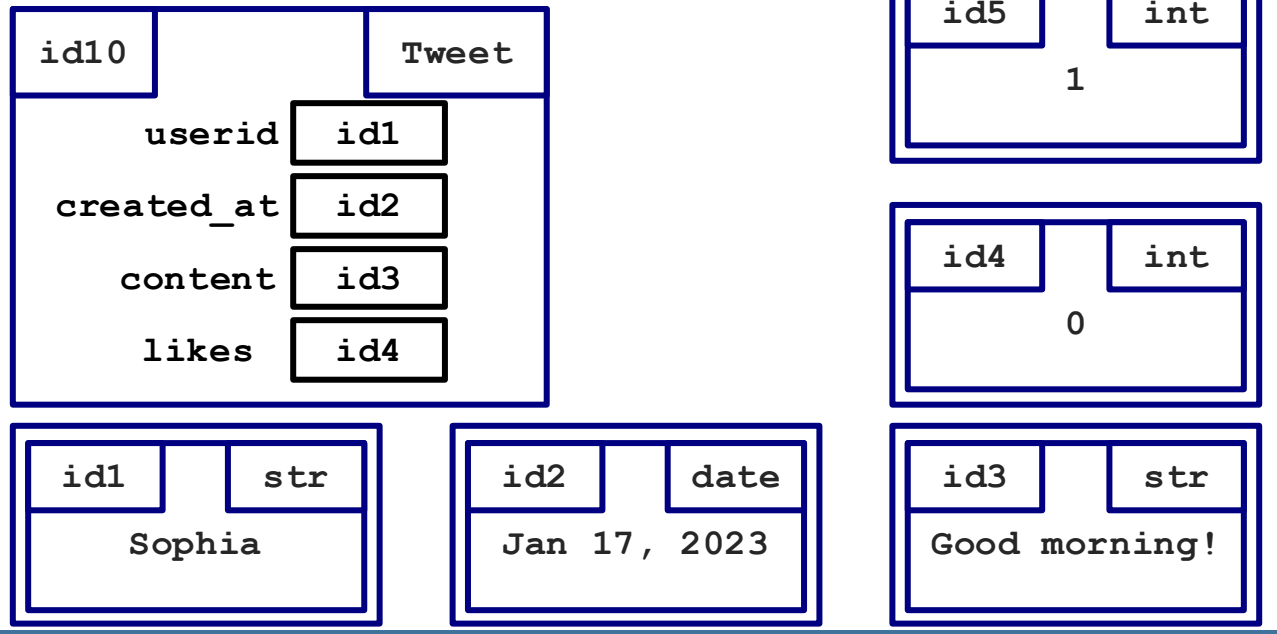


Reminder: this is our Call Stack!

```
class Tweet:
    ...
    def like(self, n: int) -> None:
        self.likes += n
```

```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```



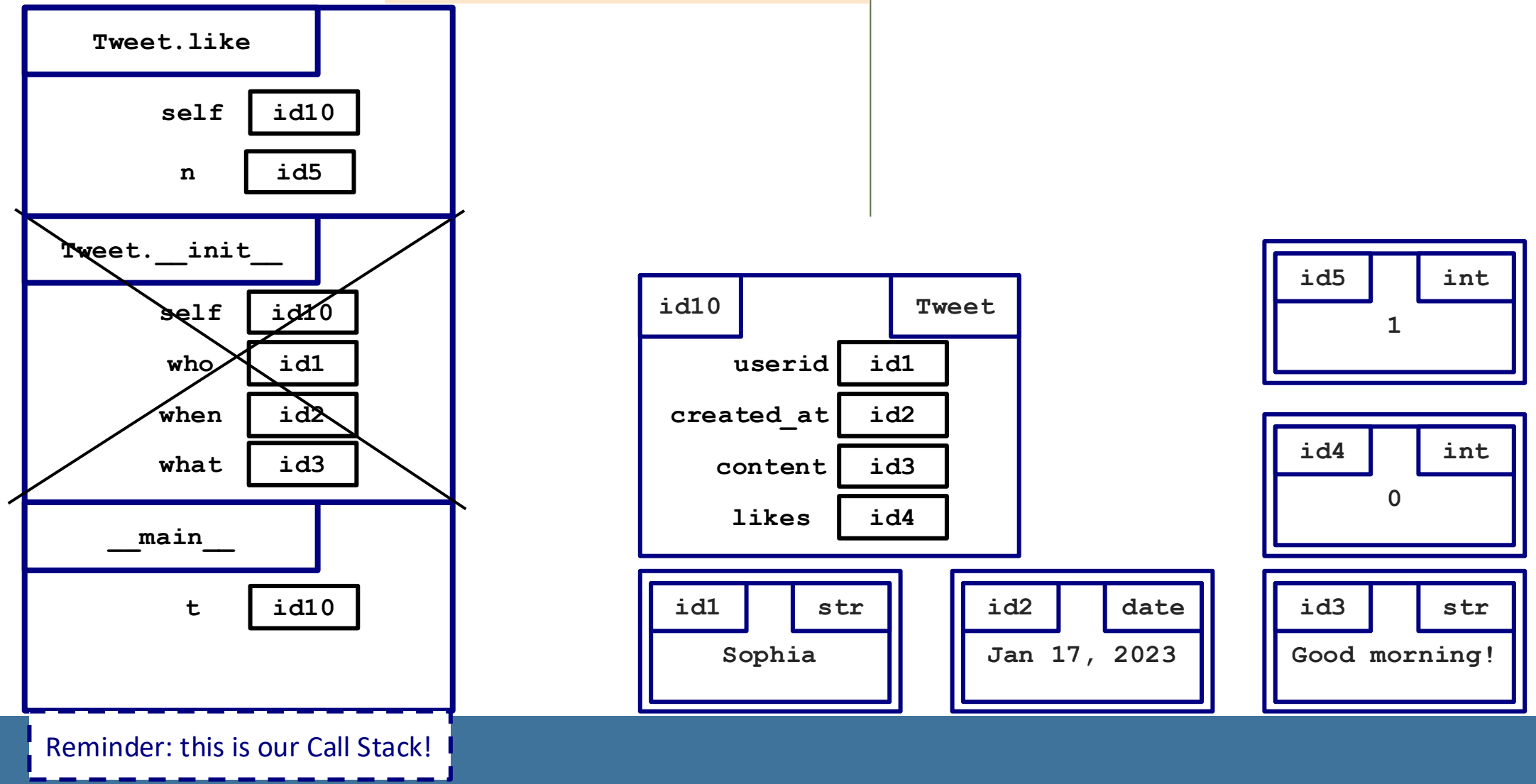


Reminder: this is our Call Stack!

```
class Tweet:
    ...
    def like(self, n: int) -> None:
        self.likes += n
```

Same as self.likes = self.likes + n

```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
t.like(1)
```



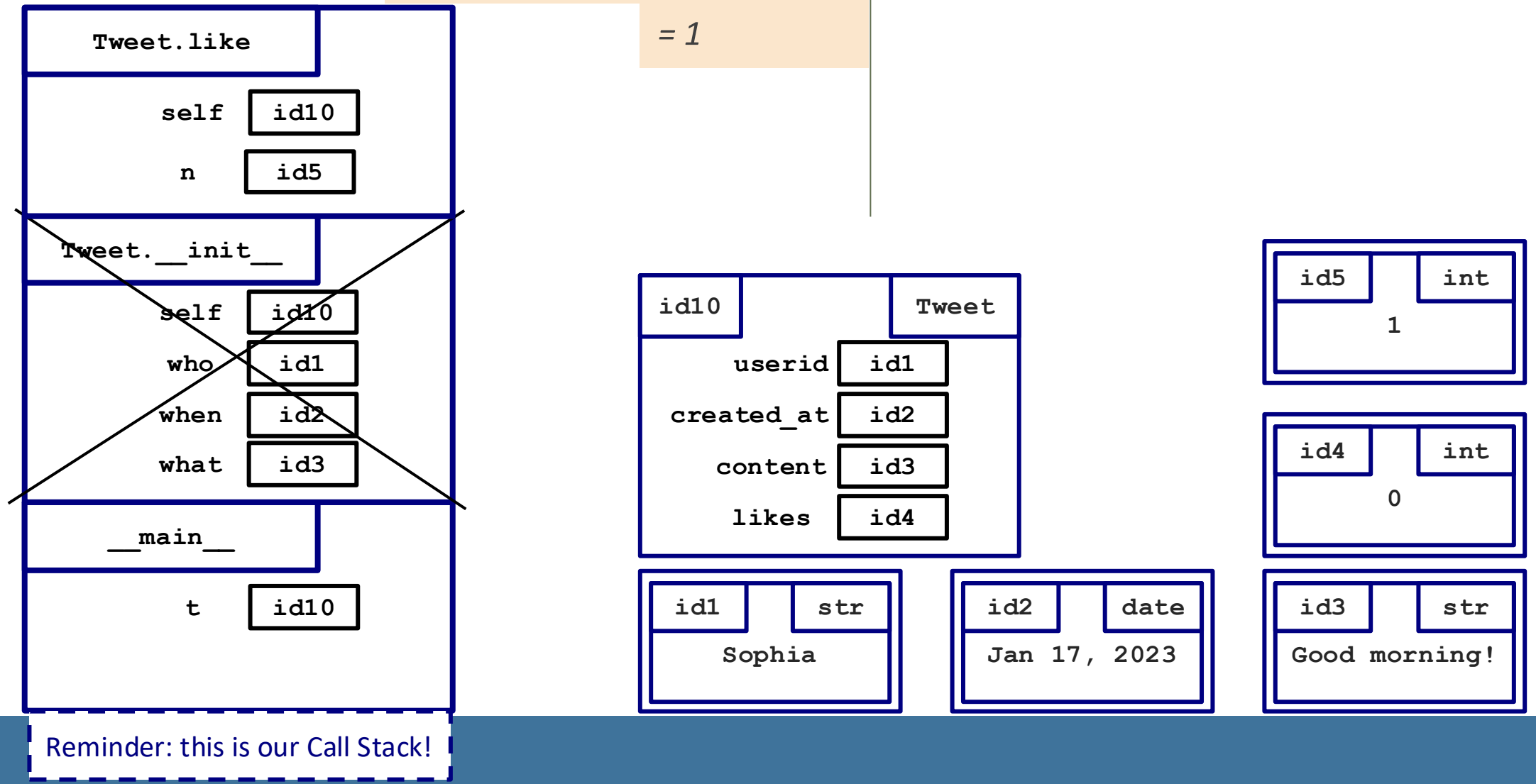
Reminder: this is our Call Stack!

```
class Tweet:
    ...
    def like(self, n: int) -> None:
        self.likes += n
```

Same as self.likes = self.likes + n

= 1

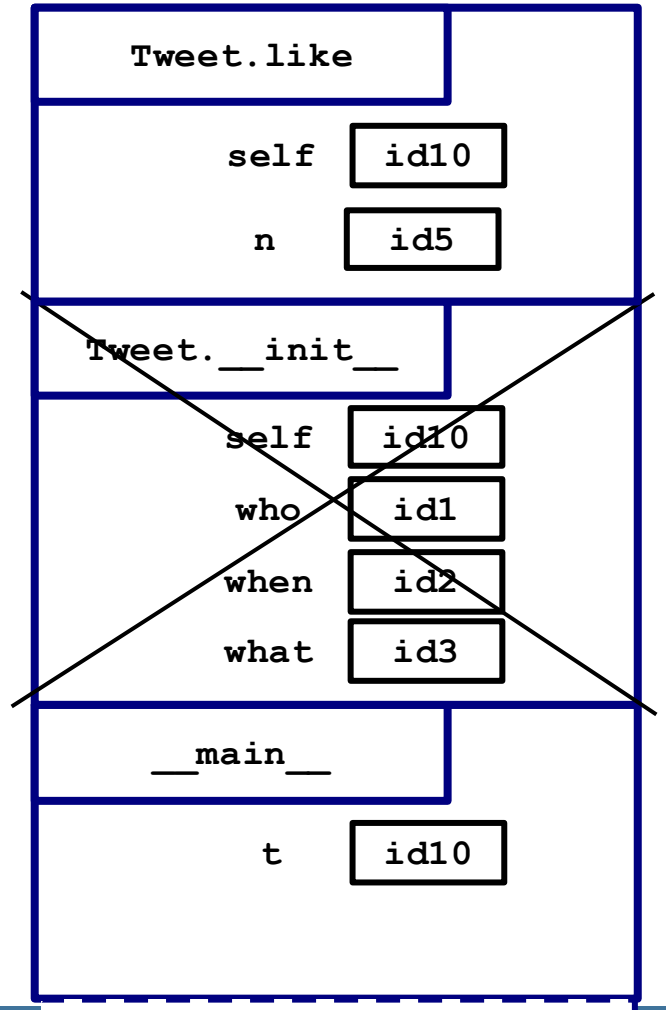
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
t.like(1)
```



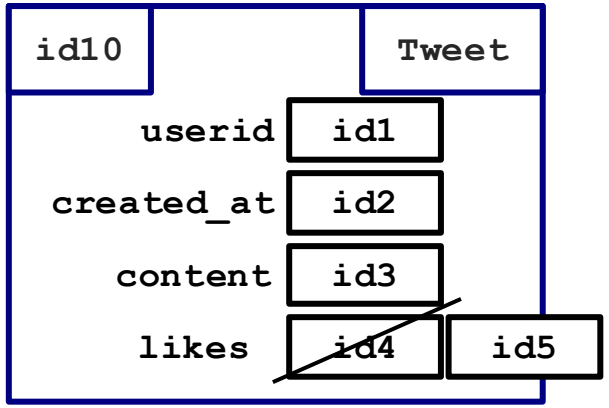
Reminder: this is our Call Stack!

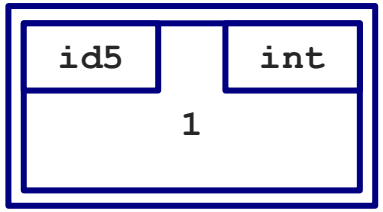
```
class Tweet:
    ...
    def like(self, n: int) -> None:
        self.likes += n
```

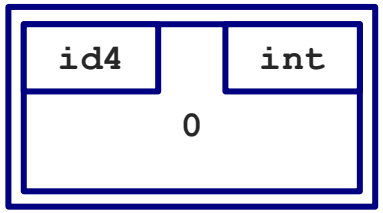
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
t.like(1)
```

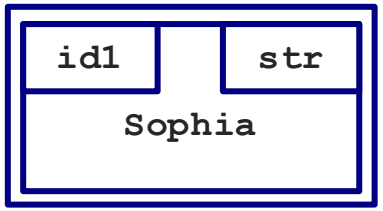


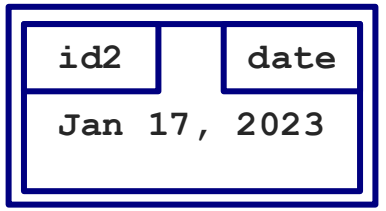
Reminder: this is our Call Stack!

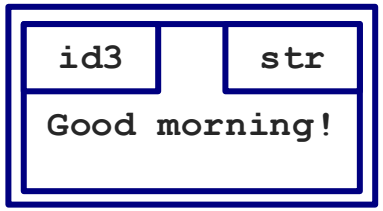






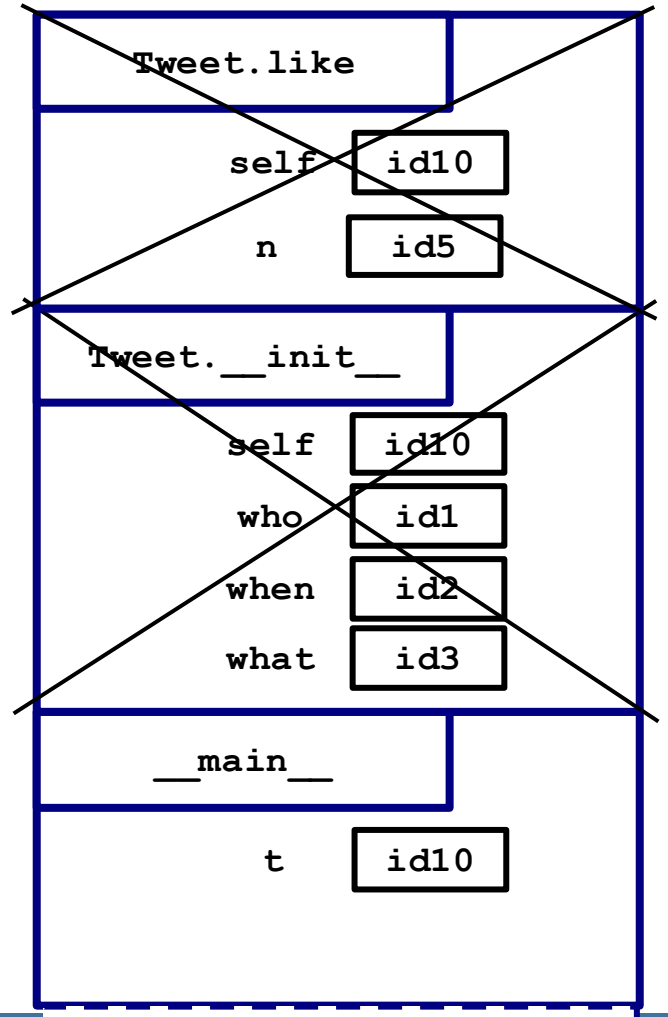




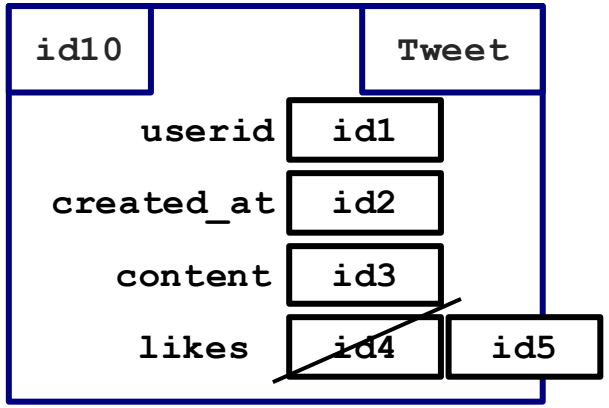


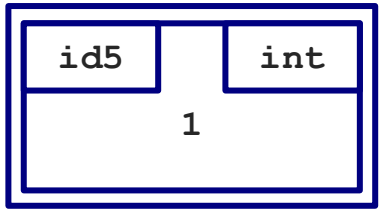
```
class Tweet:
    ...
        def like(self, n: int) -> None:
            self.likes += n
```

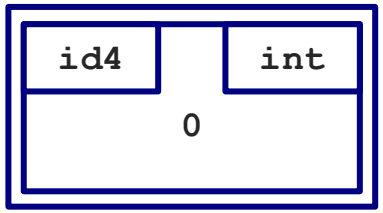
```
if __name__ == '__main__':
    t = Tweet('Sophia', date(2023, 1, 17),
              'Good morning!')
    t.like(1)
```

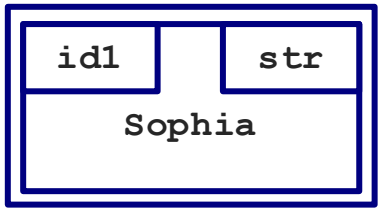


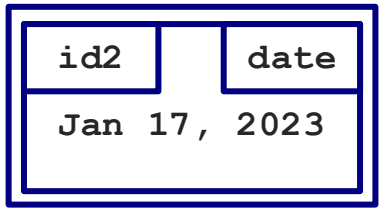
Reminder: this is our Call Stack!

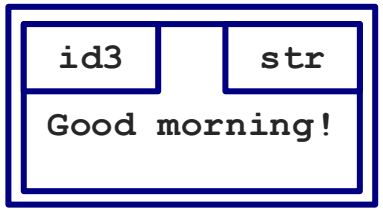












OOP1.pdf

1. Below is the initializer from an incorrect implementation of the **Spinner** class from this week's prep:

```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

Looking at the results of the class doctests, we observe that this code raises an error:

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

(i) On a separate piece of paper, draw a memory model diagram for this code.

(ii) What does this initializer actually do?

(iii) What error is raised when we run this doctest (i.e., be more specific than just `ERROR ...`).

2. Here is the documentation for the **Tweet** class you saw in Prep2, with one new method called **edit** added:

```
class Tweet:
    """A tweet, like in Twitter.

    === Attributes ===
```

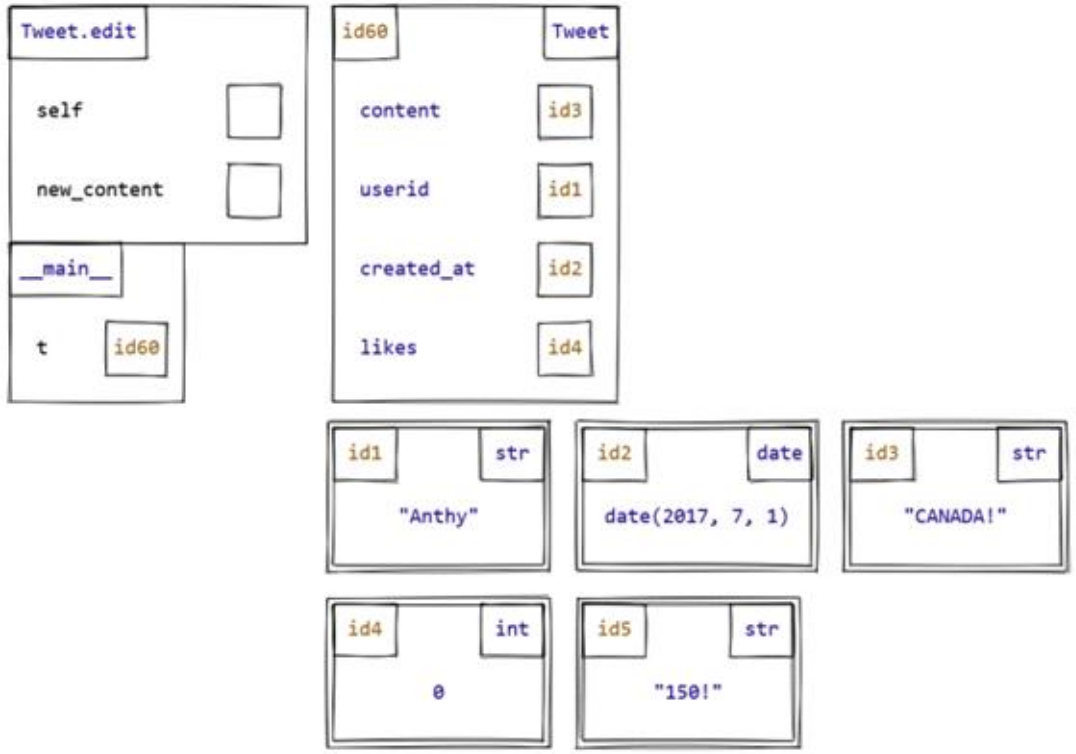
3. Here's an incorrect implementation of `Tweet.edit`:

```
class Tweet:
    def edit(self, new_content: str) -> None:
        self = Tweet(self.userid, self.created_at, new_content)
```

When we run the following code, the wrong value is displayed:

```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> t.content  # Displays 'CANADA!', not '150!'
```

Explain this problem by completing this memory model diagram showing the call to `Tweet.edit`.

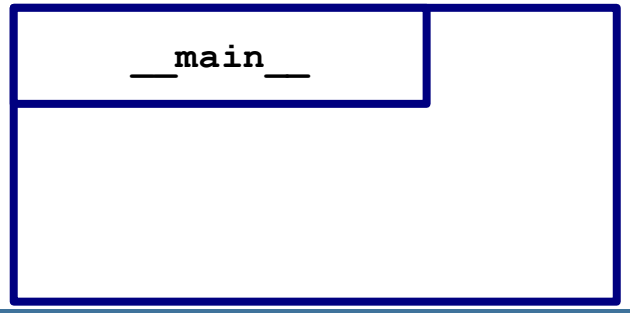


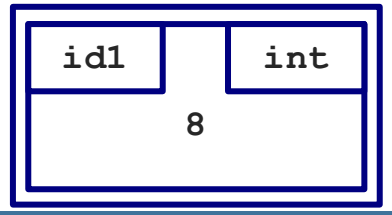
4. Implement method `Tweet.edit` correctly in the space below.

```
class Tweet:
    def edit(self, new_content: str) -> None:
```

```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

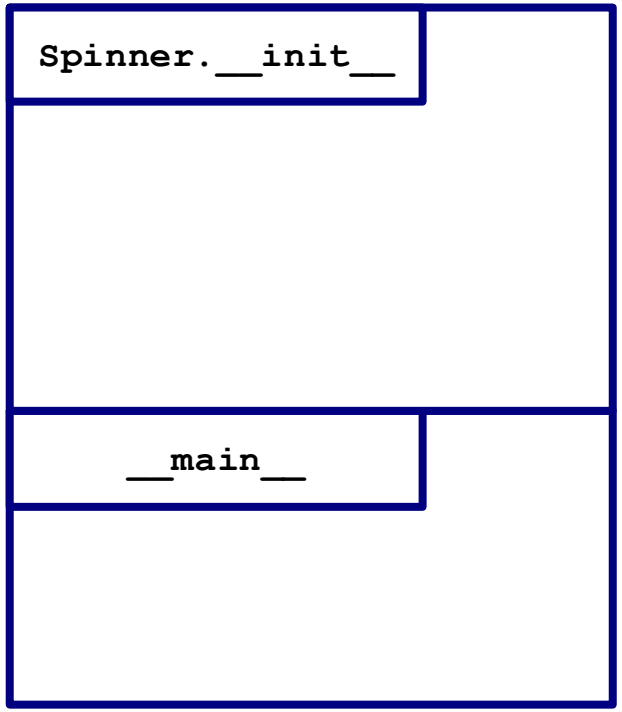
```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

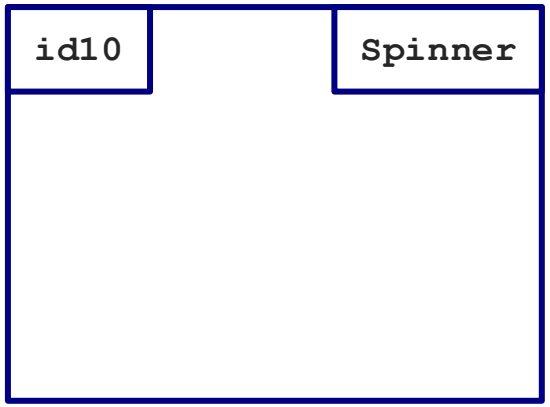


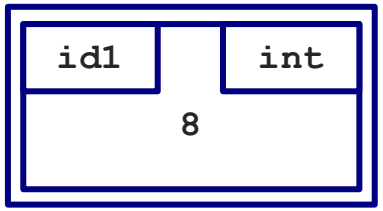


```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

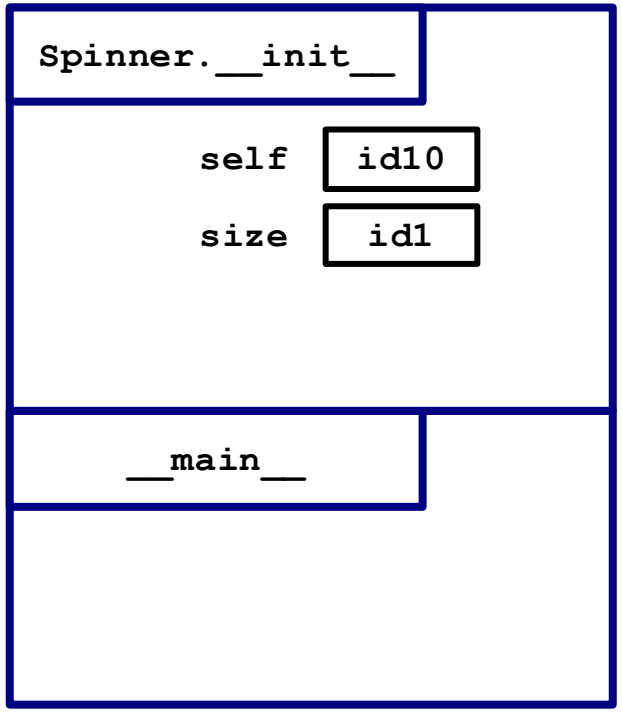


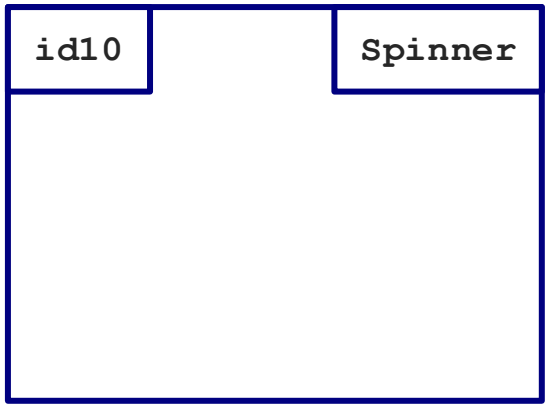


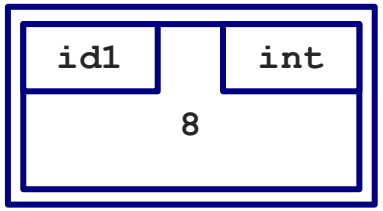


```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

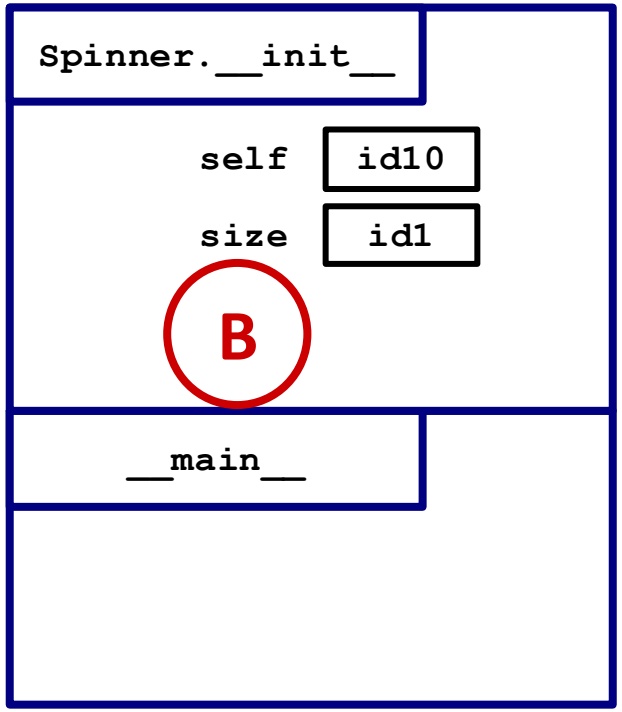


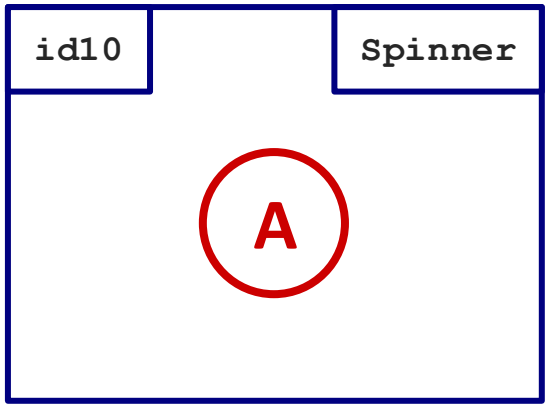


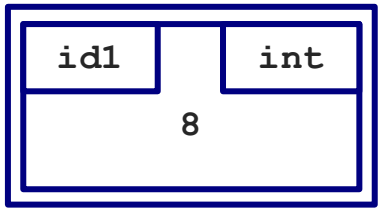


```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```



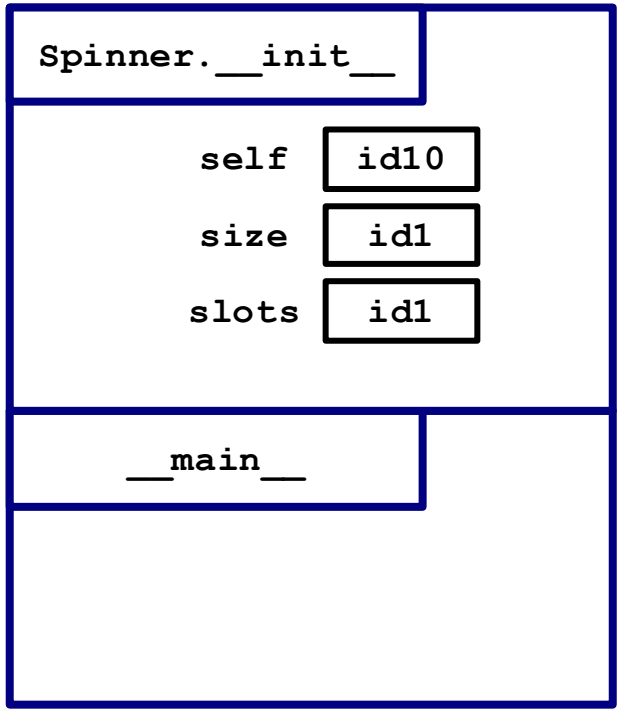


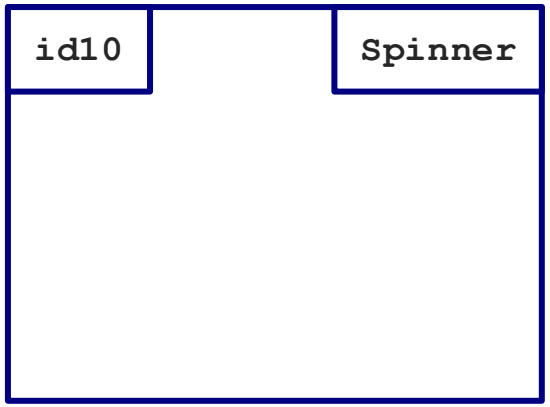


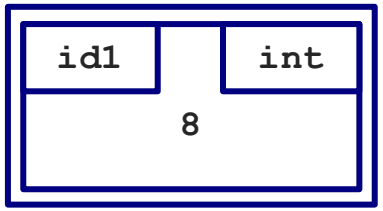
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

There's no self!

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

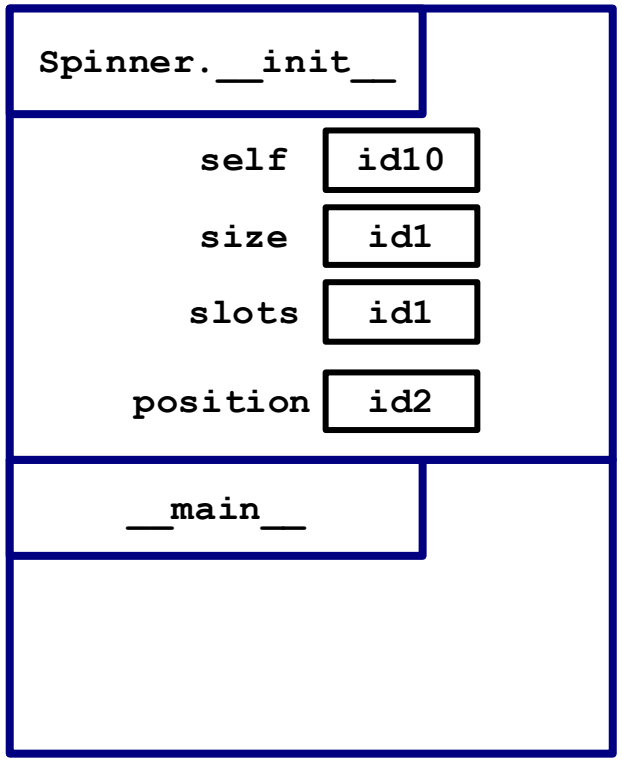


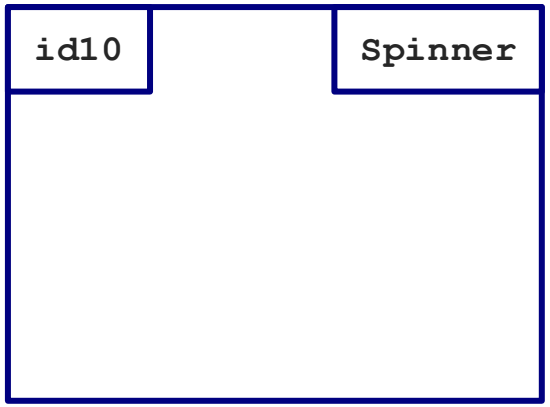


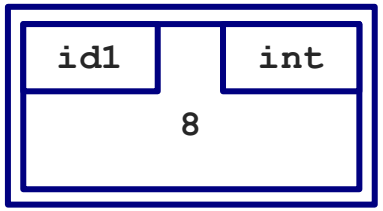


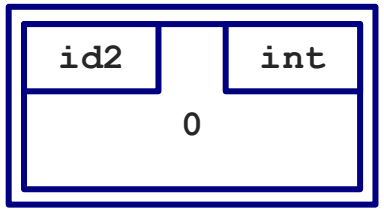
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```



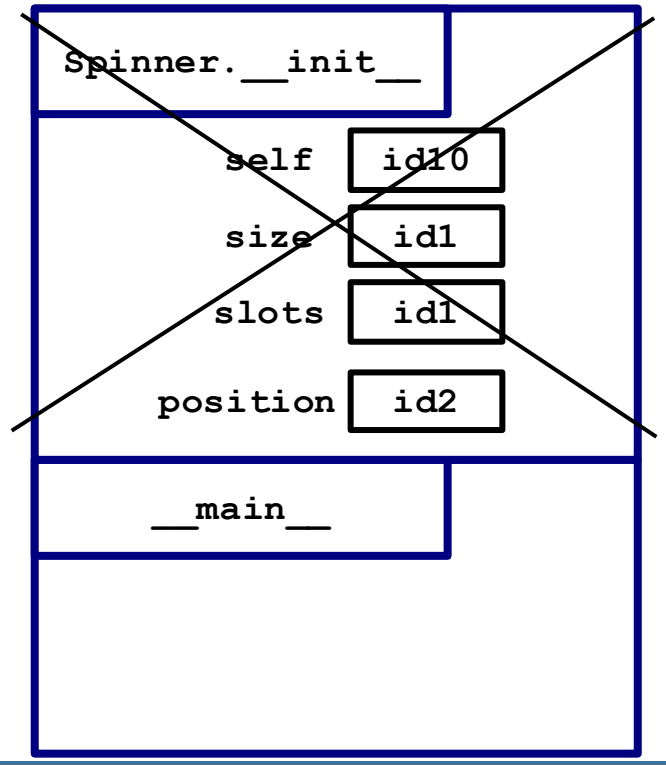


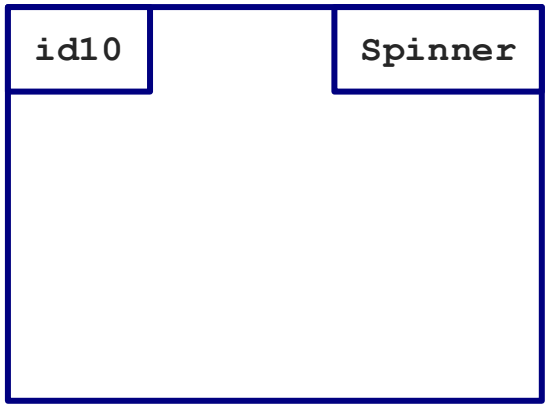


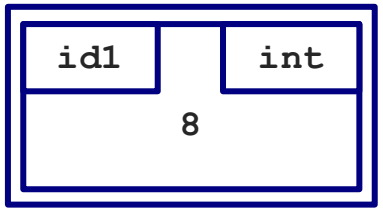


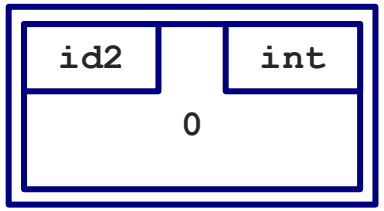
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```



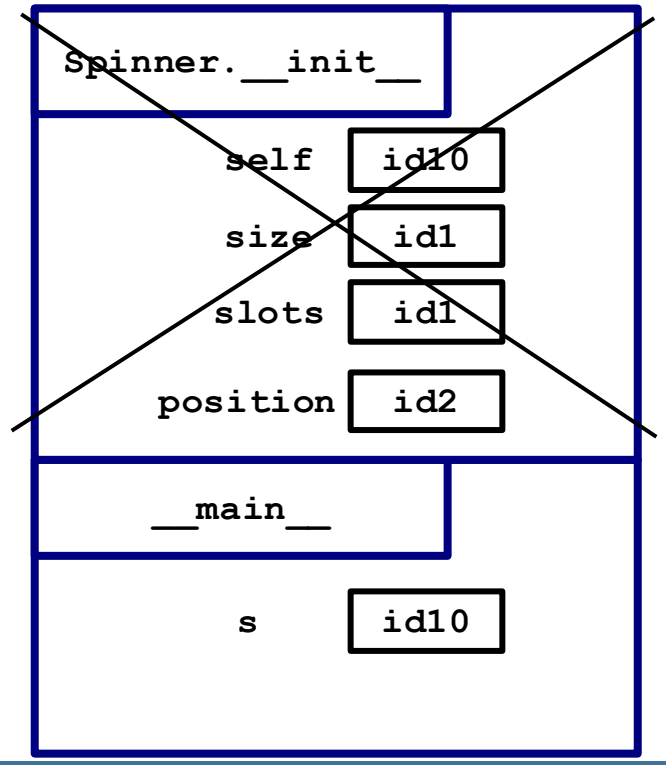


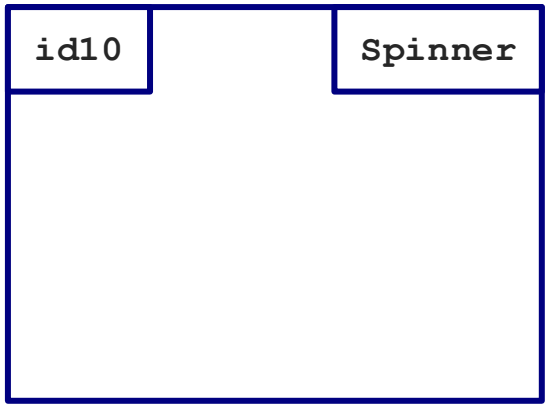


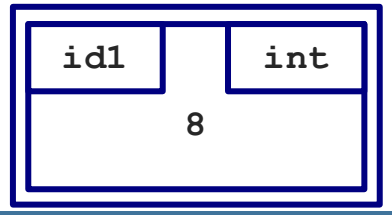


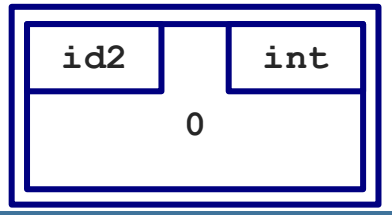
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```



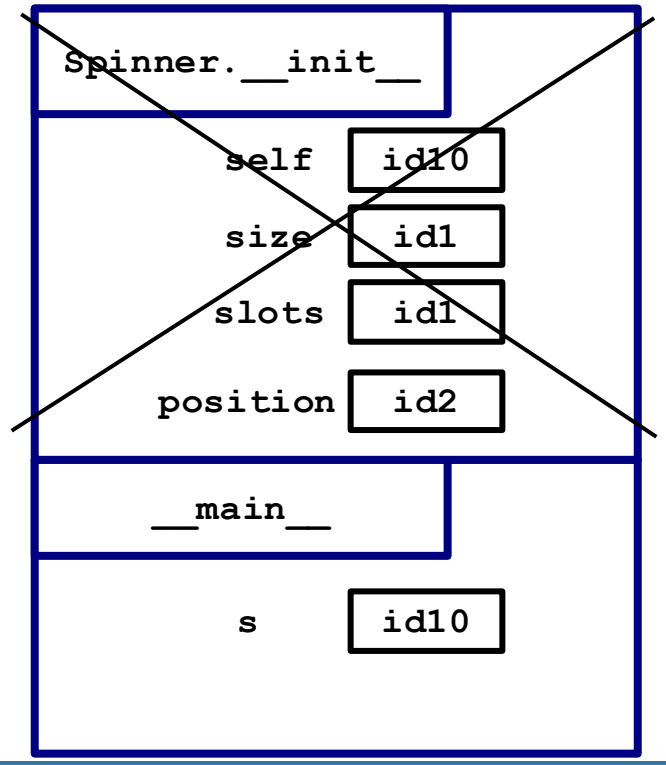


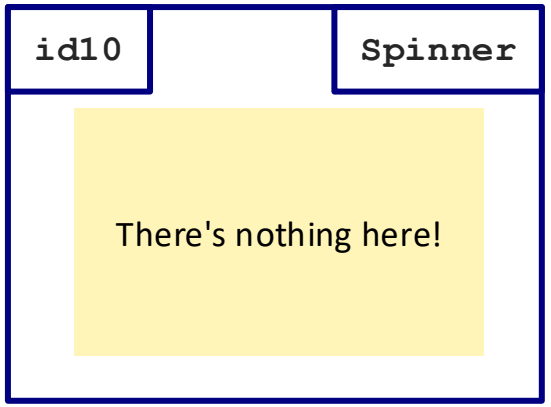


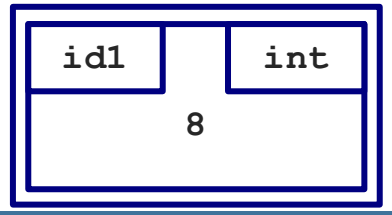


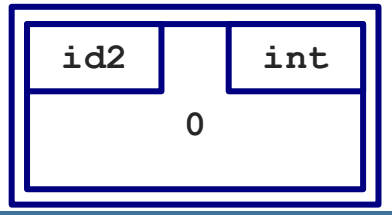
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```





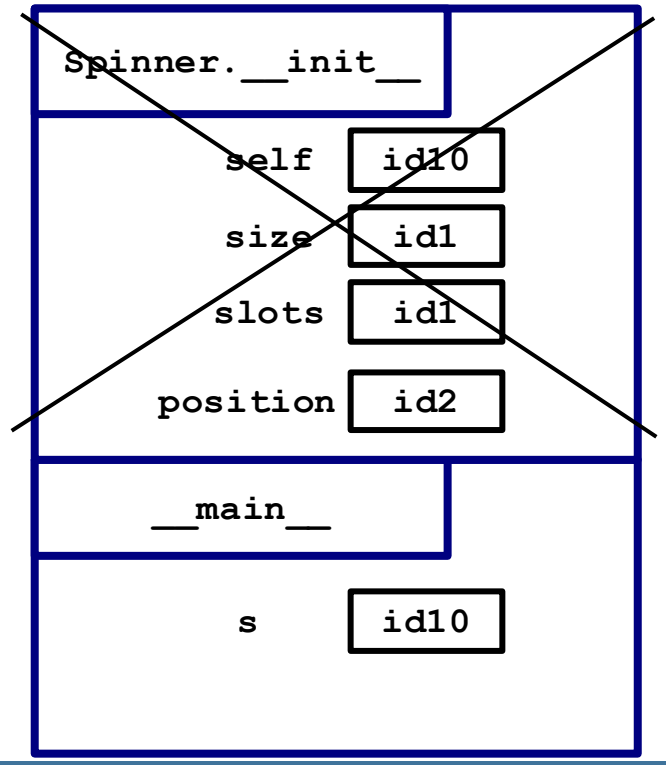


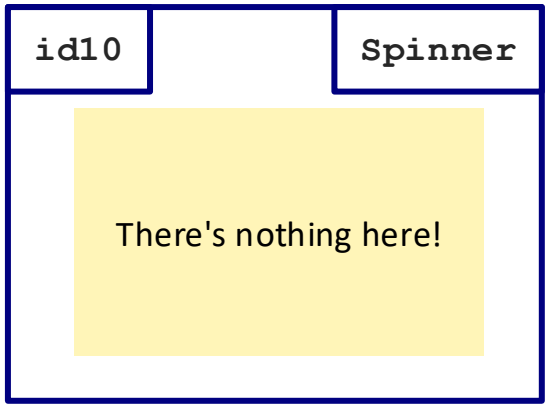


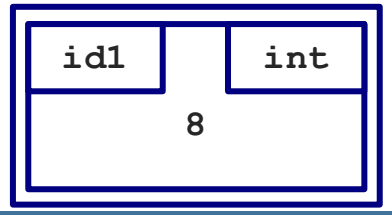
```
class Spinner:
    slots: int
    position: int

    def __init__(self, size: int) -> None:
        """Initialize a new spinner with <size> slots.

        A spinner's position always starts at 0.
        """
        slots = size
        position = 0
```

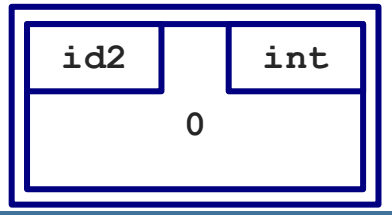
```
>>> s = Spinner(8)
>>> s.position
ERROR ...
```

AttributeError: 'Spinner' object has no attribute 'position'



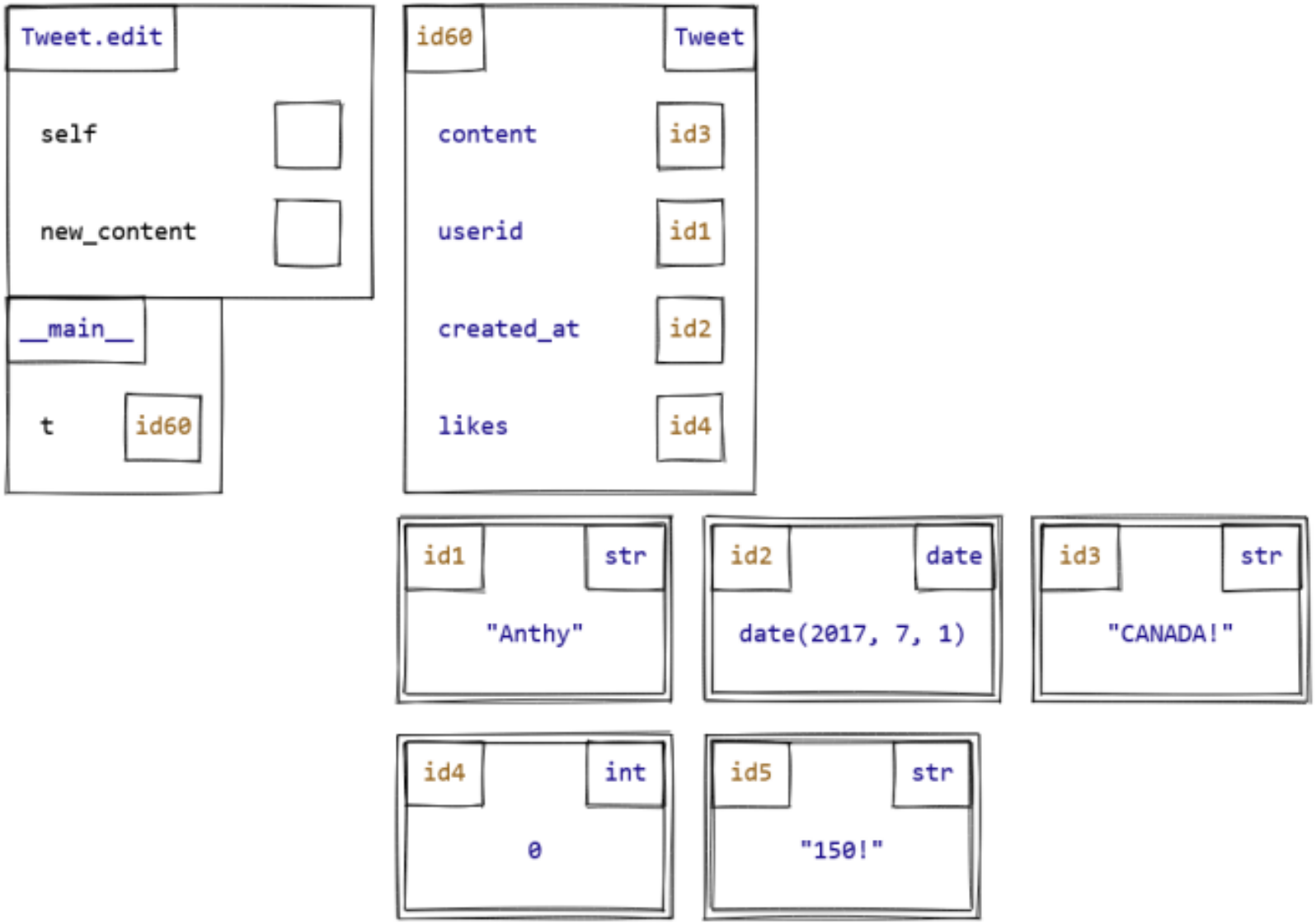






```
def edit(self, new_content: str) -> None:
    self = Tweet(old_user, old_date, new_content)
```

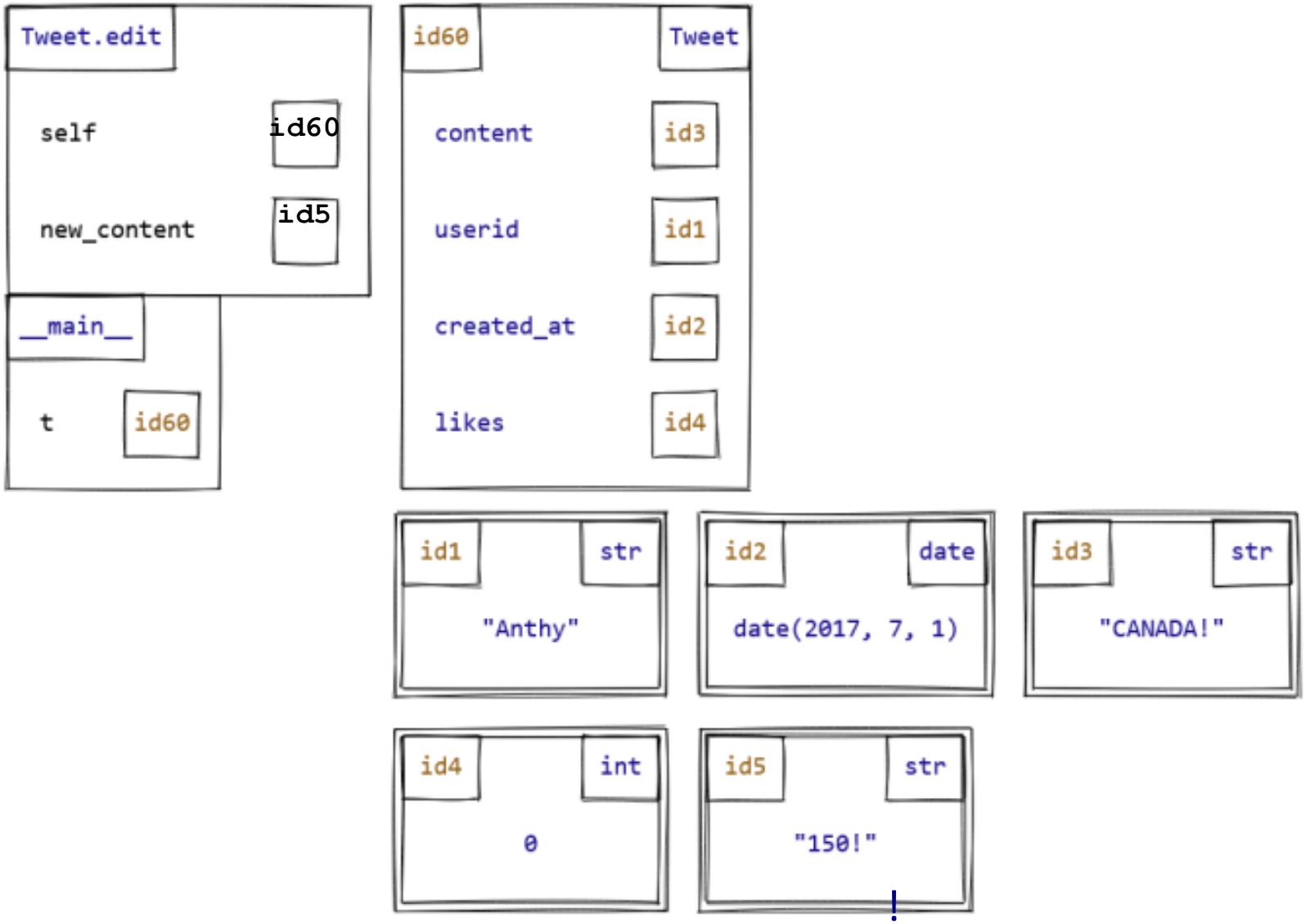
```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```



```
def edit(self, new_content: str) -> None:
```

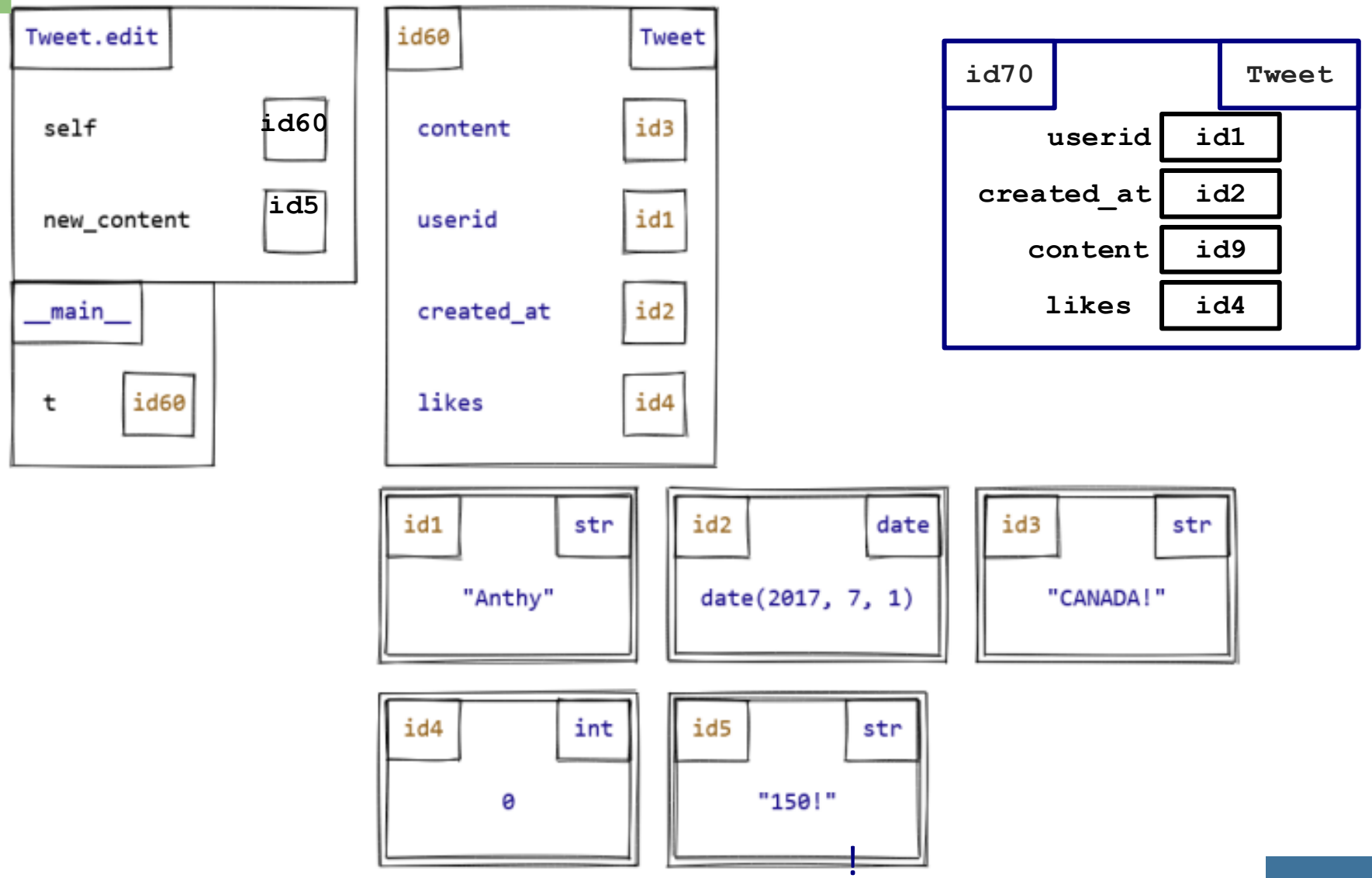
```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```

```
self = Tweet(old_user, old_date, new_content)
```



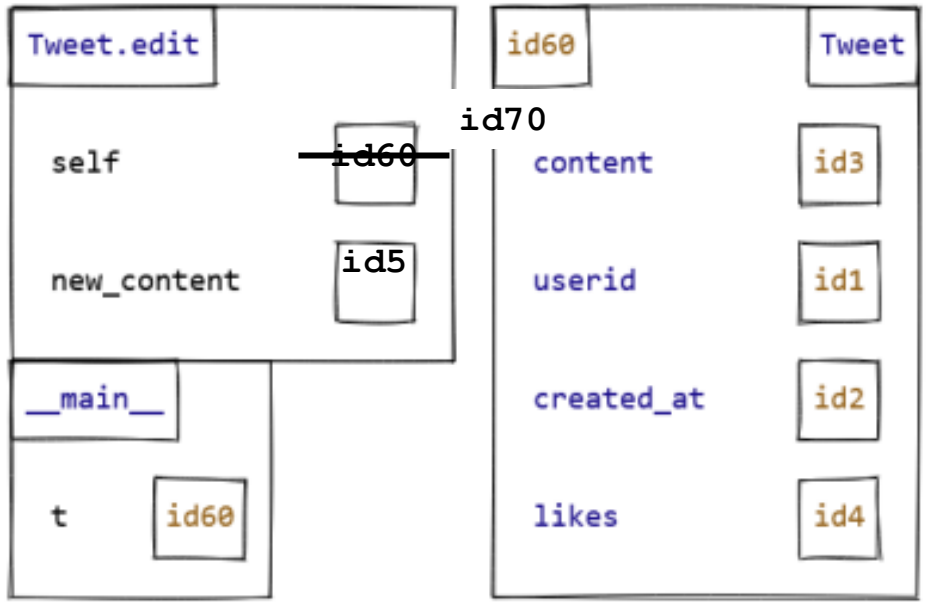
```
def edit(self, new_content: str) -> None:

    self = Tweet(old_user, old_date, new_content)
```

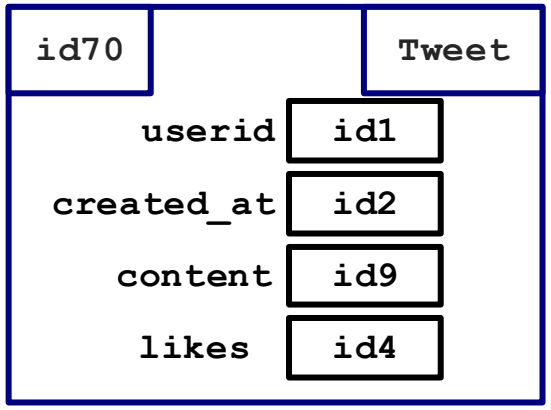
```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```

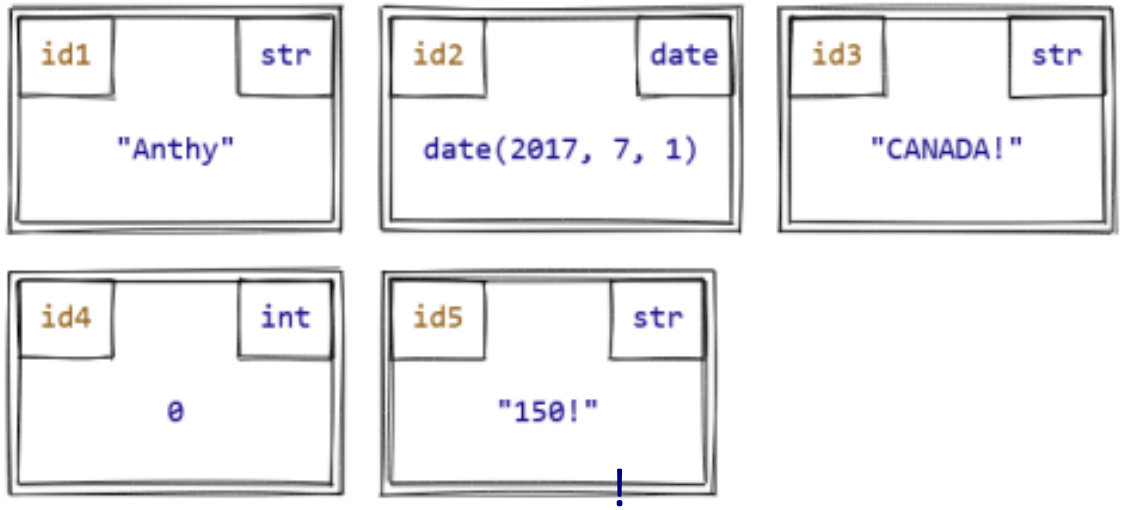


```
def edit(self, new_content: str) -> None:

    self = Tweet(old_user, old_date, new_content)
```

```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```

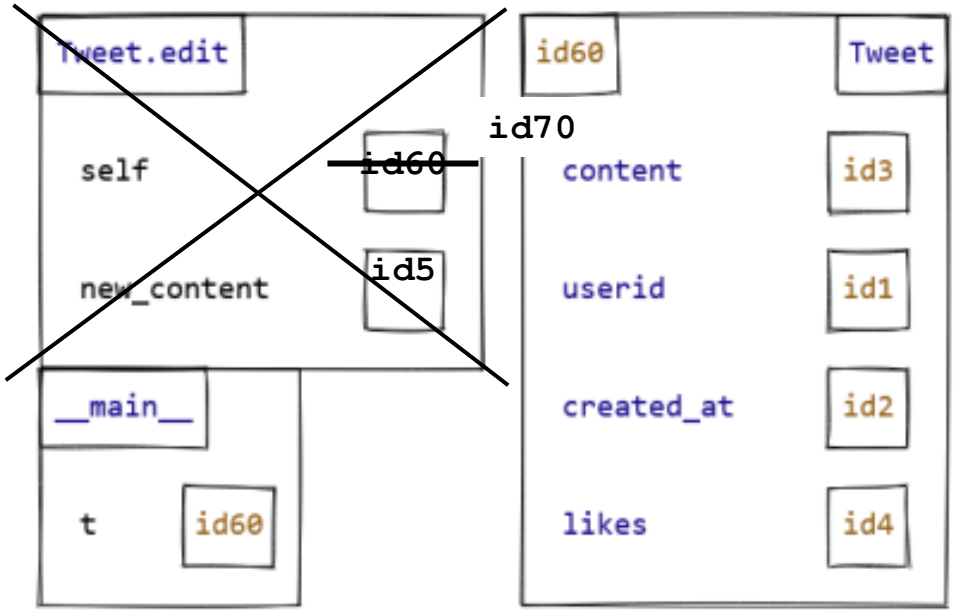


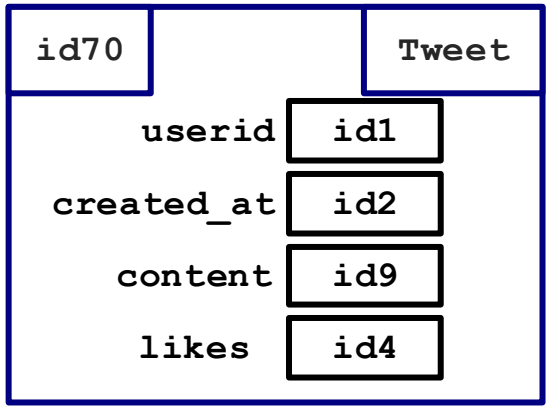


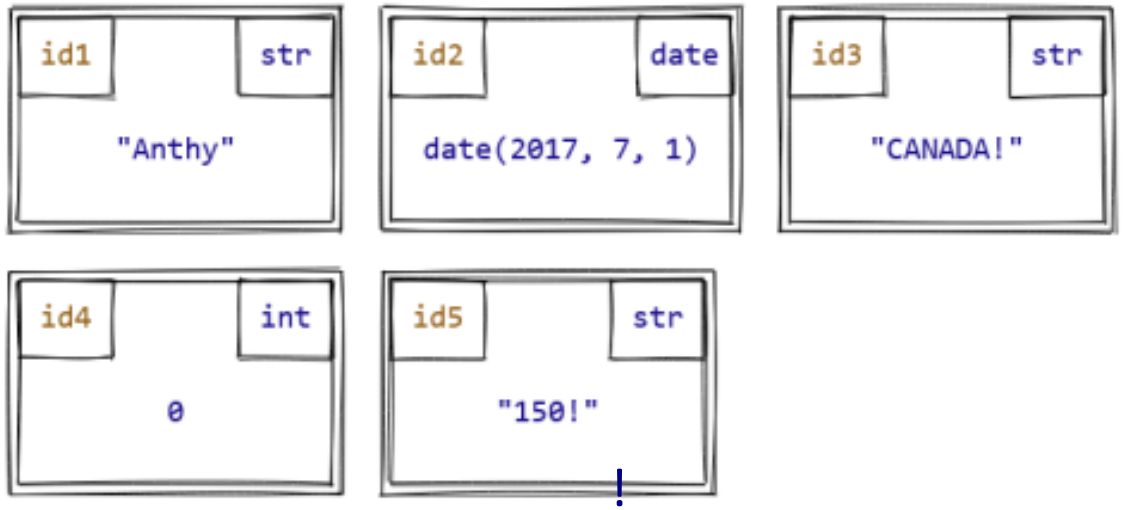


```
def edit(self, new_content: str) -> None:

    self = Tweet(old_user, old_date, new_content)
```

```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```

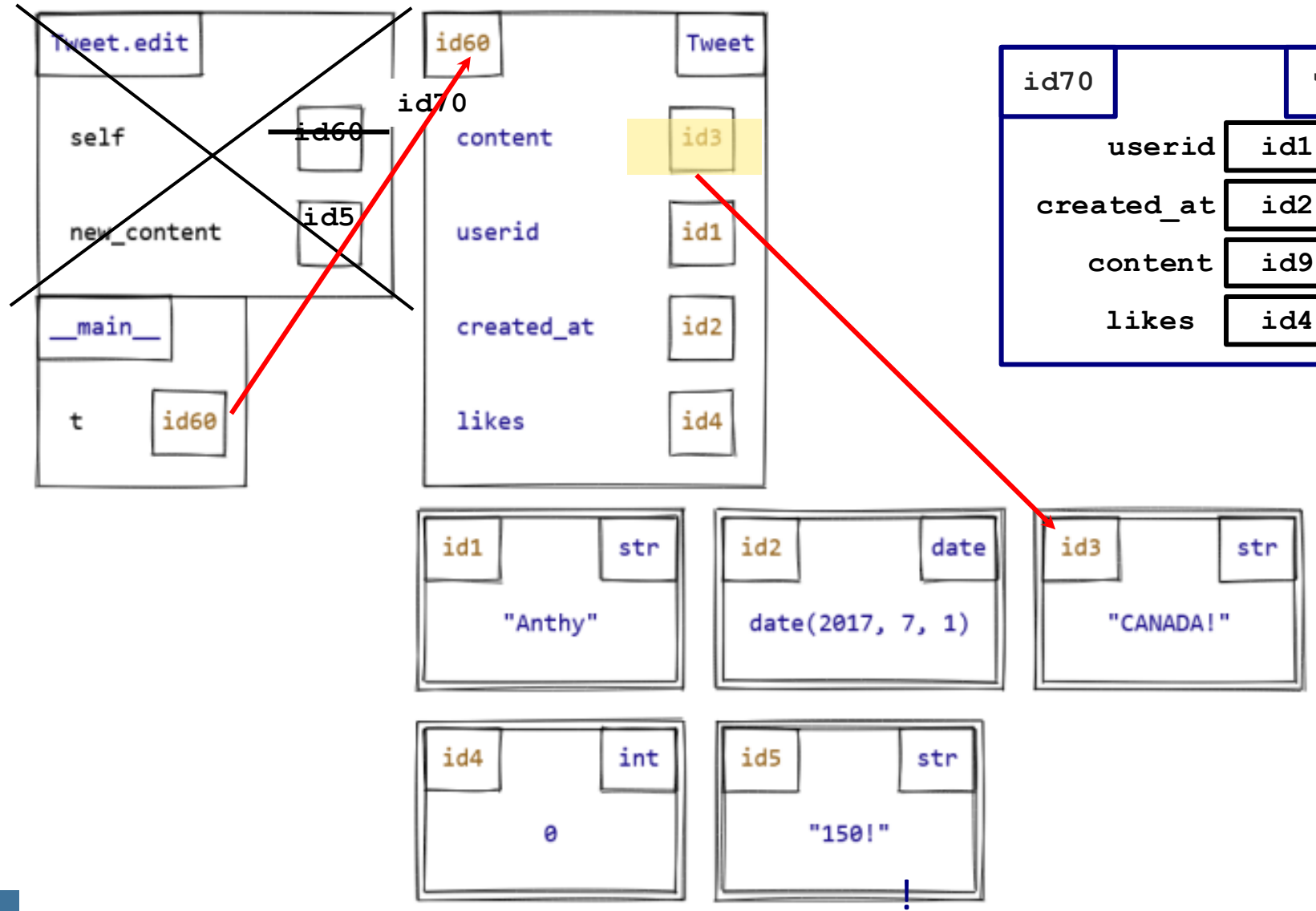






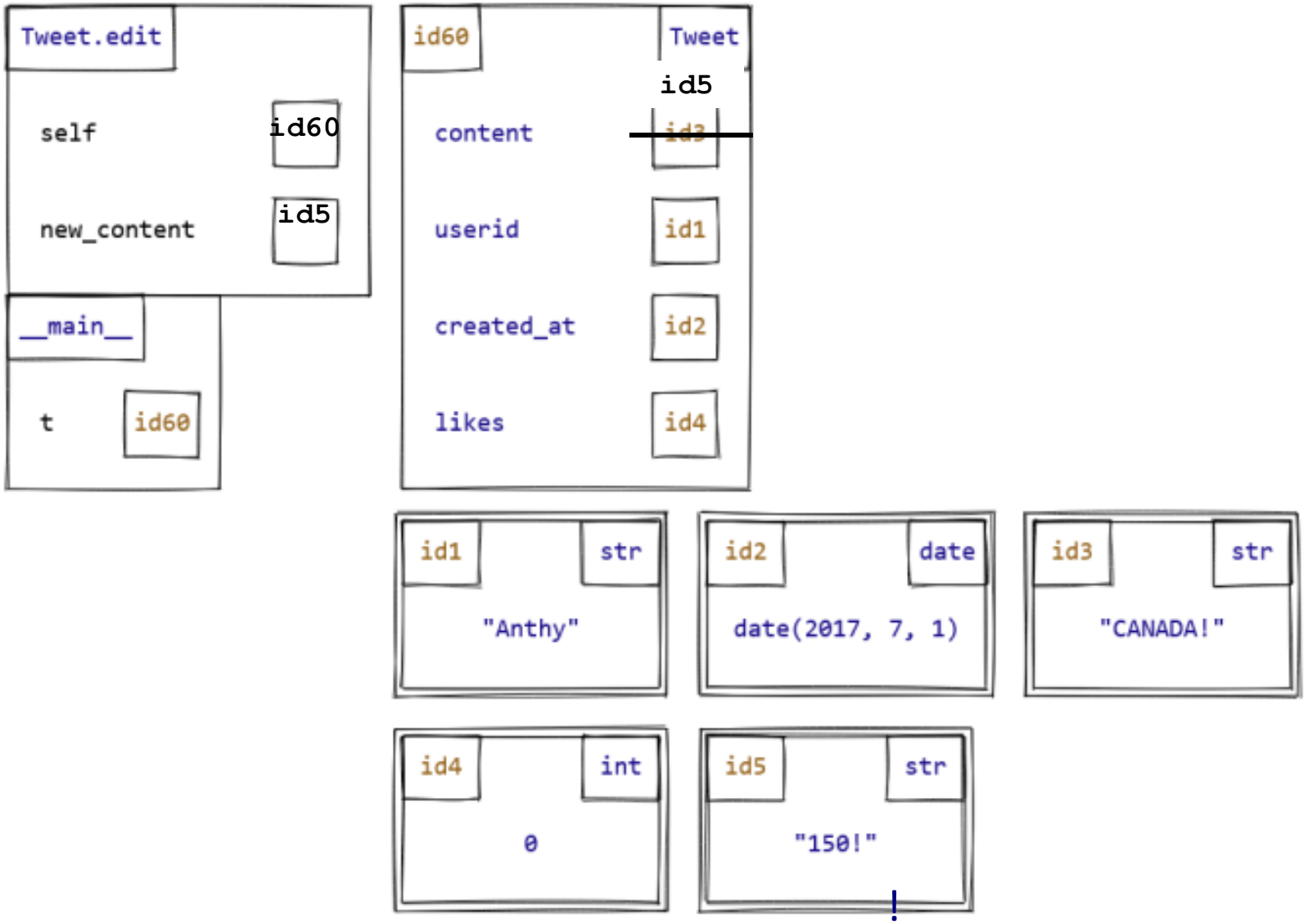
```
def edit(self, new_content: str) -> None:

    self = Tweet(old_user, old_date, new_content)
```

```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```



```
def edit(self, new_content: str) -> None:
    self.content = new_content
```

```
>>> t = Tweet('Anthy', date(2017, 7, 1), 'CANADA!')
>>> t.edit('150!')
>>> print(t.content)  # Prints 'CANADA!', not '150!'
```



Accessing an instance attribute

```
def whatever(self, x):
    y = expression
```

vs

```
    self.y = expression
```

Must use self to get at instance attributes!

Re-assigning self

```
def mutate(self, x):
    self = NewObject(x)
```

Re-assigning `self` doesn't mutate anything!

Composition of classes

HOW DO WE DESIGN CODE IN WHICH DIFFERENT CLASSES INTERACT WITH EACH OTHER?

Related Reading: 3.4 More on Designing Classes: Combining classes - composition

Users are objects too

A Twitter user has an id, a biography, and tweets (among other things).

A bear @A_single_bear · Sep 10

I wish I knew how to bark. Maybe humans would yell at me less. I am a bear.

15 228 1.7K

@A_single_bear

Hello, I am a bear.

The start of a User class

```
class User:
    """A Twitter user.

    Attributes:
    - userid: the userid of this Twitter user.
    - bio: the bio of this Twitter user.
    - tweets: a list of the tweets that this user has made.
    """
    userid: str
    bio: str
    tweets: list[Tweet]
```

User and Tweet

Composition

a relationship between two classes where instances of one class contain references to instances of the other

“has” relationship, e.g. “user has tweets”

OOP2.pdf

Here's the start of another class that will interact with class `Tweet`, and will represent a *user* of Twitter.

```
class User:
    """A Twitter user.

    Attributes:
    - userid: the userid of this Twitter user.
    - bio: the bio of this Twitter user.
    - tweets: the tweets that this user has made.
    """
    userid: str
    bio: str
    tweets: list[Tweet]
```

1. Implement an initializer for this class according to the docstring below.

```
class User:
    def __init__(self, userid: str, bio: str) -> None:
        """Initialize a new user with the given id and bio.

        The new user initially has no tweets.
        """
```

2. Implement the `User` method `tweet` according to the docstring below.

```
def tweet(self, message: str) -> None:
    """Record that this User made a tweet with the given content.

    Use date.today() to get the current date for the newly-created tweet.
    """
```

3. Suppose we want another `User` method that will record the fact that the user follows another user with a given userid. We'll used a modified version of the *Function Design Recipe* to design it.

   First, write down a sample call to this method. (Do *not* worry about how you'll implement it yet!)

4. Write a header and docstring for this method. Don't forget to include type annotations for the parameters and return value. (We've left space for the method body, but don't write it yet.)

5. Decide what attribute(s) you will use to store information about who this user follows. Then, update the class docstring and attribute type annotations (on the previous page) to record your decisions.

6. Finally, implement your new method according to its docstring.

OOP2.pdf

Here's the start of another class that will interact with class `Tweet`, and will represent a *user* of Twitter.

```
class User:
    """A Twitter user.

    Attributes:
    - userid: the userid of this Twitter user.
    - bio: the bio of this Twitter user.
    - tweets: the tweets that this user has made.
    """
    userid: str
    bio: str
    tweets: list[Tweet]
```

1. Implement an initializer for this class according to the docstring below.

```
class User:
    def __init__(self, userid: str, bio: str) -> None:
        """Initialize a new user with the given id and bio.

        The new user initially has no tweets.
        """
```

OOP2.pdf

2. Implement the `User` method `tweet` according to the docstring below.

```
def tweet(self, message: str) -> None:
    """Record that this User made a tweet with the given content.

    Use date.today() to get the current date for the newly-created tweet.
    """
```

OOP2.pdf

3. Suppose we want another `User` method that will record the fact that the user follows another user with a given userid. We'll used a modified version of the *Function Design Recipe* to design it.

   First, write down a sample call to this method. (Do *not* worry about how you'll implement it yet!)

4. Write a header and docstring for this method. Don't forget to include type annotations for the parameters and return value. (We've left space for the method body, but don't write it yet.)

5. Decide what attribute(s) you will use to store information about who this user follows. Then, update the class docstring and attribute type annotations (on the previous page) to record your decisions.

6. Finally, implement your new method according to its docstring.